File Reference: docs/build_complete_guide.py

A file-specific explanation covering purpose, owner layer, metadata, source details, implementation signals, source excerpts, and safe-change rules.

Item	Explanation
File	docs/build_complete_guide.py
Category	Python tooling
Folder role	Documentation and generated reference area. Used to explain the app and source structure.
Size	222,291 bytes
Extension	.py
MIME type	text/x-python
Text lines	3,273

Why this file exists

- Tracked repository file used by the builder, website, installer, documentation, or release process.

How it is used

Documentation and generated reference area. Used to explain the app and source structure.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	import html
Line 4	import json
Line 5	import math
Line 6	import mimetypes
Line 7	import re
Line 8	import shutil
Line 9	import subprocess
Line 10	import textwrap

API endpoints mentioned or called

Item	Explanation
/api/...	Line 103. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/code/save	Line 459. Saves source code into the local compile workspace.
/api/code/compile	Line 652. Compiles or runs a source file and returns terminal output.
/api/code/compile	Line 664. Compiles or runs a source file and returns terminal output.
/api/portfolio-ai	Line 676. Handles AI assistant questions.
/api/portfolio-ai	Line 707. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Rich text at line 14	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from PIL import Image, ImageDraw, ImageFont
Rich text at line 20	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from docx.shared import Inches, Pt, RGBColor
Rich text at line 23	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from reportlab.lib import colors
Installer at line 54	Windows setup, update, shortcut, or installation behavior. Evidence: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces release version bumps, uploads artifacts, and
Git command at line 56	Repository state, publishing, branch checks, or release automation. Evidence: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
Cloudflare or AI at line 59	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Git command at line 60	Repository state, publishing, branch checks, or release automation. Evidence: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
Cloudflare or AI at line 61	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
Cloudflare or AI at line 62	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
Installer at line 71	Windows setup, update, shortcut, or installation behavior. Evidence: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
Installer at line 73	Windows setup, update, shortcut, or installation behavior. Evidence: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
Cloudflare or AI at line 75	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
Cloudflare or AI at line 76	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",
Cloudflare or AI at line 77	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
Git command at line 78	Repository state, publishing, branch checks, or release automation. Evidence: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
Installer at line 81	Windows setup, update, shortcut, or installation behavior. Evidence: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
Compiler or simulator at line 83	Part of the Compile Code workspace or language/tool detection. Evidence: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
Security or auth at line 91	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
Git command at line 100	Repository state, publishing, branch checks, or release automation. Evidence: ("High-Level System Map", "The system has two personalities: a private builder app and a public website. The builder is where content is created and verified. The website is what recruiters see after publishing.", "Owner
Security or auth at line 102	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Private Builder Versus Public Website", "The builder includes editing controls, local draft storage, authentication checks, compiler tools, and publishing actions. The public website removes those controls and only dis
Git command at line 103	Repository state, publishing, branch checks, or release automation. Evidence: ("Local Backend Role", "The frontend cannot safely do everything by itself. The local backend handles file writes, repository operations, compiler execution, publishing checks, and generated site output.", "template-prev
Git command at line 105	Repository state, publishing, branch checks, or release automation. Evidence: ("GitHub Pages Publishing", "The target repository receives static files. GitHub Pages then serves those files over HTTPS. The builder does not run on the public site.", "Local publish mirror -> git commit -> git push ->
Cloudflare or AI at line 106	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Role", "Cloudflare can sit in front of the domain and can also host Worker endpoints such as the portfolio AI backend. DNS points the custom domain to the website host.", "Visitor DNS lookup -> Cloudflare DN

Item	Explanation
Cloudflare or AI at line 107	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI Backend Role", "The website AI should answer from portfolio context when the question is about Maurice's work and use general knowledge when the question is general. The Worker endpoint keeps API keys off the public
Installer at line 109	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer And Update Role", "The installer places the app in the per-user AppData Programs folder. Updates should replace that install instead of creating duplicate copies.", "GitHub Release -> installer download -> up
Cloudflare or AI at line 110	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Branch Model", "Development collects work. Main is the release branch. Feature branches should merge into development first. Main should receive only development.", "feature branch -> development -> main -> release wor
Security or auth at line 111	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Security Boundary", "Editing is local and authenticated. Public visitors should not get editing tools. Publishing requires GitHub write access to the target repository.", "Visitor can read public site\nOwner can edit b
Cloudflare or AI at line 119	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Cloudflare Role": "cloudflare-ai-flow",
Cloudflare or AI at line 120	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "AI Backend Role": "cloudflare-ai-flow",
Installer at line 122	Windows setup, update, shortcut, or installation behavior. Evidence: "Installer And Update Role": "release-pipeline",
Cloudflare or AI at line 123	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Branch Model": "release-pipeline",
Git command at line 128	Repository state, publishing, branch checks, or release automation. Evidence: ("git status", "Shows what branch you are on and whether files are changed, staged, or clean.", "Run it before edits, before commits, and before merges."),
Git command at line 129	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch development", "Moves your working copy to the development branch.", "Use development as the integration branch before anything goes to main."),
Git command at line 130	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch -c codex/example-change", "Creates a new feature branch from the branch you are currently on.", "Use this before changing the builder so development stays stable."),
Git command at line 131	Repository state, publishing, branch checks, or release automation. Evidence: ("git add <file>", "Stages a changed file for the next commit.", "Only stage files that belong to the current change."),
Git command at line 132	Repository state, publishing, branch checks, or release automation. Evidence: ("git commit -m \"message\"", "Records staged changes as a named checkpoint.", "Write messages that explain why the change exists."),
Git command at line 133	Repository state, publishing, branch checks, or release automation. Evidence: ("git push origin <branch>", "Uploads your branch to GitHub.", "This lets GitHub run workflows and enables pull requests."),
Git command at line 134	Repository state, publishing, branch checks, or release automation. Evidence: ("git pull --ff-only", "Updates the current branch only if Git can fast-forward safely.", "Avoids surprise merge commits."),
Git command at line 135	Repository state, publishing, branch checks, or release automation. Evidence: ("git fetch --tags --force", "Downloads branch and tag metadata from GitHub.", "Used before checking whether a release tag already exists."),
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: ("pnpm install --frozen-lockfile", "Installs Node dependencies exactly as pinned in pnpm-lock.yaml.", "Makes release builds repeatable."),
Installer at line 139	Windows setup, update, shortcut, or installation behavior. Evidence: ("pnpm run installer", "Builds the NSIS Windows installer.", "Use this when testing installation and update behavior."),
Cloudflare or AI at line 142	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("wrangler deploy", "Deploys the Cloudflare Worker backend.", "Use it when the AI endpoint code changes."),
Compiler or simulator at line 143	Part of the Compile Code workspace or language/tool detection. Evidence: ("gcc file.c -o app.exe", "Compiles C source into a Windows executable.", "The builder uses similar logic when C tools are available."),
Compiler or simulator at line 145	Part of the Compile Code workspace or language/tool detection. Evidence: ("javac Main.java", "Compiles Java source into .class bytecode.", "Java has a compile step before running."),
Compiler or simulator at line 146	Part of the Compile Code workspace or language/tool detection. Evidence: ("java Main", "Runs compiled Java bytecode.", "The builder shows this output in the terminal panel."),
Compiler or simulator at line 147	Part of the Compile Code workspace or language/tool detection. Evidence: ("python script.py", "Runs Python source directly.", "Python is interpreted by default."),
Compiler or simulator at line 148	Part of the Compile Code workspace or language/tool detection. Evidence: ("node script.js", "Runs JavaScript through Node.js.", "Useful for builder scripts and JavaScript examples."),
Compiler or simulator at line 149	Part of the Compile Code workspace or language/tool detection. Evidence: ("iverilog -g2012 design.sv -o sim.vvp", "Compiles Verilog/SystemVerilog into an Icarus simulation file.", "Prepares HDL for simulation."),

Item	Explanation
Compiler or simulator at line 150	Part of the Compile Code workspace or language/tool detection. Evidence: ("vvp sim.vvp", "Runs Icarus Verilog simulation output.", "This is how HDL terminal output appears."),
Git command at line 151	Repository state, publishing, branch checks, or release automation. Evidence: ("git push --dry-run origin main", "Tests whether GitHub would accept a push without changing the remote.", "Used during authorization checks."),
Git command at line 152	Repository state, publishing, branch checks, or release automation. Evidence: ("git remote -v", "Shows which GitHub repository the local folder talks to.", "Important for publishing targets."),
Git command at line 153	Repository state, publishing, branch checks, or release automation. Evidence: ("git branch --show-current", "Prints the active branch name.", "The builder must know whether it is pushing the right branch."),
Git command at line 154	Repository state, publishing, branch checks, or release automation. Evidence: ("git tag --list builder-v*", "Lists builder release tags.", "Release workflows use this to prevent duplicate releases."),
Installer at line 155	Windows setup, update, shortcut, or installation behavior. Evidence: ("Get-ChildItem", "PowerShell command that lists files and folders.", "Used to inspect workspaces and installer output."),
Process execution at line 157	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: ("Start-Process", "PowerShell command that starts another program.", "The updater uses detached process launching."),
Cloudflare or AI at line 162	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Profile identity and contact details", "Front-page hero text", "Fun facts block", "Portfolio areas", "Project categories", "Project creation flow", "Project overview", "Design sections", "Simulation sections", "Files an
Installer at line 167	Windows setup, update, shortcut, or installation behavior. Evidence: ("The app does not open", "Check whether the installed executable exists under AppData Local Programs. If it does, start it directly. If it fails, reinstall from the latest GitHub Release."),
Installer at line 168	Windows setup, update, shortcut, or installation behavior. Evidence: ("Update does not restart", "The update handoff must close Electron, run the installer, and then launch the new executable. Check the updater log and whether another builder process is still running."),
Security or auth at line 169	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("GitHub authentication succeeds but builder cannot push", "Authentication and repository freshness are different. Pull or load from target first if the remote branch is ahead."),
Local storage at line 171	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Website changes show on desktop but not phone", "Check cache, confirm the published files changed, and verify the mobile renderer is using the same projects.json and CSS."),
Cloudflare or AI at line 172	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI endpoint unavailable", "Verify the Worker route, environment secrets, Cloudflare deployment, and browser network errors."),
Local storage at line 173	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Compiler takes too long", "Check whether the first run is installing or detecting tools. Cached runs should be faster after the first successful compile."),
Compiler or simulator at line 174	Part of the Compile Code workspace or language/tool detection. Evidence: ("SystemVerilog output missing", "Check that Icarus Verilog and vvp are available and that the file is a simulation testbench or has a runnable top module."),
Rich text at line 175	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Right-click formatting does not apply", "Confirm the active editor saved its selection before the menu changed font, size, or color."),
Rich text at line 176	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Links are treated as formulas", "URL detection should win before formula detection. Pasted http, https, and www links must become normal links."),
Installer at line 178	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer reports an old installation", "The installer searches per-user and legacy locations. Remove stale install records only after confirming the active app path."),
Git command at line 186	Repository state, publishing, branch checks, or release automation. Evidence: "What HTML does", "What CSS does", "What JavaScript does", "What JSON does", "What Markdown does", "What YAML does", "What TOML does", "What an executable is", "What an installer is", "What a portable app is", "What a lo
Git command at line 192	Repository state, publishing, branch checks, or release automation. Evidence: ("Electron vs pure web builder", "Electron gives desktop file access and packaging. A pure web builder would be easier to access anywhere, but browser security would block many local file, compiler, and Git operations.")
Installer at line 194	Windows setup, update, shortcut, or installation behavior. Evidence: ("Per-user install vs Program Files", "Per-user installs update without admin prompts. Program Files feels traditional but causes UAC friction and update failures."),
Git command at line 197	Repository state, publishing, branch checks, or release automation. Evidence: ("Git authentication cache vs always prompt", "Caching reduces annoyance. Always prompting is stricter but makes normal publishing painful."),
Cloudflare or AI at line 198	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Worker AI vs browser-only AI", "Worker AI hides secrets and can access server-side providers. Browser-only AI would expose keys or be limited to local browser models."),
Git command at line 213	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "Git target repo\ncommit + push", 1040, 140, 250, 135, "#FEF3C7"),

Item	Explanation
Cloudflare or AI at line 216	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("ai", "Cloudflare Worker\nportfolio AI endpoint", 1040, 565, 250, 130, "#CCFBF1"),
Git command at line 222	Repository state, publishing, branch checks, or release automation. Evidence: ("backend", "git", "publishes"),
Git command at line 223	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "site", "hosts files"),
Cloudflare or AI at line 225	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("site", "ai", "chat API"),
Git command at line 237	Repository state, publishing, branch checks, or release automation. Evidence: ("system", "Filesystem, Git,\ncompilers, updater", 1045, 250, 300, 140, "#DCFCE7"),
Installer at line 279	Windows setup, update, shortcut, or installation behavior. Evidence: ("release", "GitHub Release\ninstaller + portable", 1015, 370, 300, 130, "#EDE9FE"),
Compiler or simulator at line 297	Part of the Compile Code workspace or language/tool detection. Evidence: ("lang", "Language profile\nC, C++, Java,\nPython, JS, HDL", 705, 150, 260, 145, "#FEF3C7"),
Security or auth at line 299	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("scope", "HDL testbench\n+ signal scope", 1030, 315, 250, 125, "#DCFCE7"),

JSON/YAML-style keys detected

Item	Explanation
.github/workflows/build-windows-builder.yml	Line 54: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces releas
.github/workflows/main-branch-gate.yml	Line 55: ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main and rejects pull requests into main unless the source branch is development.",
.gitignore	Line 56: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
.nojekyll	Line 57: ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files publish exactly as generated.",
.well-known/security.txt	Line 58: ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners to find responsible disclosure information.",
BRANCHING.md	Line 59: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Backgrounds/.gitkeep	Line 60: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
README.md	Line 61: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
_headers	Line 62: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
assets/apple-touch-icon.png	Line 63: "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or compatible launch surfaces.",
assets/favicon-16.png	Line 64: "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
assets/favicon-192.png	Line 65: "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
assets/favicon-32.png	Line 66: "assets/favicon-32.png": "Standard browser favicon size.",
assets/favicon-48.png	Line 67: "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
assets/favicon.ico	Line 68: "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
assets/omb-app-icon-256.png	Line 69: "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
assets/omb-app-icon-512.png	Line 70: "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
assets/omb-app-icon.ico	Line 71: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
assets/omb-mark.png	Line 72: "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",
build/installer.nsh	Line 73: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
builder-rich-future-sections.js	Line 74: "builder-rich-future-sections.js": "Builder-side helper code for richer future portfolio sections and section defaults.",

Item	Explanation
cloudflare/README.md	Line 75: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
cloudflare/portfolio-ai-worker.js	Line 76: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",
cloudflare/wrangler.toml	Line 77: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
docs/.gitkeep	Line 78: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
electronics-search.js	Line 79: "electronics-search.js": "Electronics vocabulary and search support used by the website search behavior.",
index.html	Line 80: "index.html": "Public website HTML shell that recruiters and visitors load in the browser.",
installer-notes.txt	Line 81: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
main.cjs	Line 82: "main.cjs": "Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.",
package.json	Line 83: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
pnpm-lock.yaml	Line 84: "pnpm-lock.yaml": "Pinned dependency lockfile so installs and GitHub Actions use the same dependency versions.",
pnpm-workspace.yaml	Line 85: "pnpm-workspace.yaml": "pnpm workspace declaration for the repository.",
portfolio-README.md	Line 86: "portfolio-README.md": "README for the generated public portfolio website rather than the builder application.",
project-templates.json	Line 87: "project-templates.json": "Appearance template definitions used by projects to control visual feel rather than content.",
projects.json	Line 88: "projects.json": "Public project catalog consumed by the website after parsing builder data.",
robots.txt	Line 89: "robots.txt": "Crawler instruction file for search engines.",
script.js	Line 90: "script.js": "Public website JavaScript for navigation, portfolio rendering, search, AI UI, and interactive project windows.",
server.mjs	Line 91: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
styles.css	Line 92: "styles.css": "Public website CSS for layout, contact area, projects, responsive behavior, AI panel, and mobile/desktop rendering.",
template-preview.css	Line 93: "template-preview.css": "Builder application CSS for dialogs, windows, rich editors, compile code, dark mode, guide windows, and local previews.",
template-preview.html	Line 94: "template-preview.html": "Builder HTML shell loaded inside the Electron app and local preview runtime.",
template-preview.js	Line 95: "template-preview.js": "Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.",
High-Level System Map	Line 115: "High-Level System Map": "system-overview",
Why Electron Was Chosen	Line 116: "Why Electron Was Chosen": "electron-runtime",
Portfolio Parser Role	Line 117: "Portfolio Parser Role": "builder-to-site-files",
GitHub Pages Publishing	Line 118: "GitHub Pages Publishing": "builder-to-site-files",
Cloudflare Role	Line 119: "Cloudflare Role": "cloudflare-ai-flow",
AI Backend Role	Line 120: "AI Backend Role": "cloudflare-ai-flow",
Compiler Workspace Role	Line 121: "Compiler Workspace Role": "compile-code-flow",
Installer And Update Role	Line 122: "Installer And Update Role": "release-pipeline",
Branch Model	Line 123: "Branch Model": "release-pipeline",
name	Line 206: "name": "system-overview",
title	Line 207: "title": "Private Builder To Public Portfolio",
nodes	Line 208: "nodes": [
edges	Line 218: "edges": [
name	Line 229: "name": "electron-runtime",

Item	Explanation
title	Line 230: "title": "Electron Runtime Inside The Desktop App",
nodes	Line 231: "nodes": [
edges	Line 239: "edges": [
name	Line 249: "name": "builder-to-site-files",
title	Line 250: "title": "How Builder Edits Become Website Files",
nodes	Line 251: "nodes": [
edges	Line 260: "edges": [
name	Line 271: "name": "release-pipeline",
title	Line 272: "title": "Builder Release Pipeline",
nodes	Line 273: "nodes": [
edges	Line 282: "edges": [
name	Line 292: "name": "compile-code-flow",
title	Line 293: "title": "Compile Code Workspace",
nodes	Line 294: "nodes": [
edges	Line 304: "edges": [
name	Line 315: "name": "tool-build-map",
title	Line 316: "title": "What Tool Builds What",
nodes	Line 317: "nodes": [
edges	Line 326: "edges": [
name	Line 337: "name": "file-communication-map",
title	Line 338: "title": "How Key Files Communicate",
nodes	Line 339: "nodes": [
edges	Line 350: "edges": [
name	Line 363: "name": "cloudflare-ai-flow",

Event handlers and user interactions

Item	Explanation
Line 464	"sectionContent.addEventListener('click', async (event) => {\n const button = event.target.closest('[data-compile-run]');\n if (button) await compileActiveFile(project, file);\n}"
Line 1951	if ".addEventListener(" in line:

Function-by-function detail

tracked_files (docs/build_complete_guide.py, lines 901-905)

tracked_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 901-905 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references run, strip, and splitlines. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 903: return [line.strip() for line in result.stdout.splitlines() if line.strip()].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
901: def tracked_files() -> list[str]:
902:     result = subprocess.run(["git", "ls-files"], cwd=REPO, text=True, capture_output=True, check=True)
903:     return [line.strip() for line in result.stdout.splitlines() if line.strip()]
904:
905:
```

is_generated_reference_file (docs/build_complete_guide.py, lines 929-932)

is_generated_reference_file named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 929-932 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 930: return repo_file in GENERATED_REFERENCE_FILES or repo_file.startswith(GENERATED_REFERENCE_PREFIXES).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
929: def is_generated_reference_file(repo_file: str) -> bool:
930:     return repo_file in GENERATED_REFERENCE_FILES or repo_file.startswith(GENERATED_REFERENCE_PREFIXES)
931:
932:
```

primary_tracked_files (docs/build_complete_guide.py, lines 933-936)

primary_tracked_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 933-936 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references is_generated_reference_file. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 934: return [file for file in files if not is_generated_reference_file(file)].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
933: def primary_tracked_files(files: list[str]) -> list[str]:
934:     return [file for file in files if not is_generated_reference_file(file)]
935:
936:
```

important_code_files (docs/build_complete_guide.py, lines 960-964)

important_code_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 960-964 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references set, and is_generated_reference_file. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 962: return [file for file in IMPORTANT_CODE_FILES if file in available and not is_generated_reference_file(file)].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
960: def important_code_files(files: list[str]) -> list[str]:
961:     available = set(files)
```

```
  962:     return [file for file in IMPORTANT_CODE_FILES if file in available and not
is_generated_reference_file(file)]
  963:
  964:
```

load_package (docs/build_complete_guide.py, lines 1208-1214)

load_package loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1208-1214 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references loads, and read_text. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1210: return json.loads((REPO / "package.json").read_text(encoding="utf-8")); line 1212: return {}.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1208: def load_package() -> dict:
1209:     try:
1210:         return json.loads((REPO / "package.json").read_text(encoding="utf-8"))
1211:     except Exception:
1212:         return {}
1213:
1214:
```

load_font (docs/build_complete_guide.py, lines 1215-1222)

load_font loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1215-1222 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Path, exists, truetype, str, and load_default. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1219: return ImageFont.truetype(str(font_path), size=size); line 1220: return ImageFont.load_default().

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1215: def load_font(size: int, bold: bool = False) -> ImageFont.ImageFont:
1216:     font_name = "arialbd.ttf" if bold else "arial.ttf"
1217:     font_path = Path(r"C:\Windows\Fonts") / font_name
1218:     if font_path.exists():
1219:         return ImageFont.truetype(str(font_path), size=size)
1220:     return ImageFont.load_default()
1221:
1222:
```

wrap_for_box (docs/build_complete_guide.py, lines 1223-1242)

wrap_for_box named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1223-1242 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references str, splitlines, split, append, and getbbox. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1240: return lines.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1223: def wrap_for_box(label: str, font: ImageFont.ImageFont, max_width: int) -> list[str]:
1224:     lines: list[str] = []
1225:     for part in str(label).splitlines():
1226:         words = part.split()
```

```
1227:         if not words:
1228:             lines.append("")
1229:             continue
1230:         current = words[0]
1231:         for word in words[1:]:
1232:             candidate = f"{current} {word}"
1233:             bbox = font.getbbox(candidate)
1234:             if bbox[2] - bbox[0] <= max_width:
1235:                 current = candidate
1236:             else:
1237:                 lines.append(current)
1238:                 current = word
1239:         lines.append(current)
1240:     return lines
1241:
1242:
```

draw_arrow (docs/build_complete_guide.py, lines 1243-1255)

draw_arrow named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1243-1255 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references line, atan2, int, cos, sin, and polygon. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1243: def draw_arrow(draw: ImageDraw.ImageDraw, start: tuple[int, int], end: tuple[int, int], color: str,
width: int = 4) -> None:
1244:     draw.line([start, end], fill=color, width=width)
1245:     angle = math.atan2(end[1] - start[1], end[0] - start[0])
1246:     length = 18
1247:     spread = 0.55
1248:     points = [
1249:         end,
1250:         (int(end[0] - length * math.cos(angle - spread)), int(end[1] - length * math.sin(angle -
spread))),
1251:         (int(end[0] - length * math.cos(angle + spread)), int(end[1] - length * math.sin(angle +
spread))),
1252:     ]
1253:     draw.polygon(points, fill=color)
1254:
1255:
```

edge_points (docs/build_complete_guide.py, lines 1256-1271)

edge_points named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1256-1271 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references abs. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1269: return start, end.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1256: def edge_points(source: tuple[int, int, int, int], target: tuple[int, int, int, int]) -> tuple[tuple[int,
int], tuple[int, int]]:
1257:     sx1, sy1, sx2, sy2 = source
1258:     tx1, ty1, tx2, ty2 = target
1259:     sc = ((sx1 + sx2) // 2, (sy1 + sy2) // 2)
1260:     tc = ((tx1 + tx2) // 2, (ty1 + ty2) // 2)
1261:     dx = tc[0] - sc[0]
1262:     dy = tc[1] - sc[1]
1263:     if abs(dx) >= abs(dy):
1264:         start = (sx2 if dx >= 0 else sx1, sc[1])
1265:         end = (tx1 if dx >= 0 else tx2, tc[1])
1266:     else:
1267:         start = (sc[0], sy2 if dy >= 0 else sy1)
1268:         end = (tc[0], ty1 if dy >= 0 else ty2)
1269:     return start, end
1270:
```

```
1271:
```

make_diagrams (docs/build_complete_guide.py, lines 1272-1321)

make_diagrams named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1272-1321 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, load_font, Draw, rounded_rectangle, text, line, edge_points, draw_arrow, getbbox, hypot, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1319: return diagram_paths.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1272: def make_diagrams() -> dict[str, Path]:
1273:     DIAGRAM_DIR.mkdir(parents=True, exist_ok=True)
1274:     title_font = load_font(38, bold=True)
1275:     box_font = load_font(24, bold=True)
1276:     label_font = load_font(18, bold=False)
1277:     diagram_paths: dict[str, Path] = {}
1278:
1279:     for spec in DIAGRAM_SPECS:
1280:         image = Image.new("RGB", (1600, 900), "#F8FBFD")
1281:         draw = ImageDraw.Draw(image)
1282:         draw.rounded_rectangle((24, 24, 1576, 876), radius=26, outline="#B7D7EA", width=3,
fill="#FFFFFF")
1283:         draw.text((60, 48), spec["title"], fill="#0B2545", font=title_font)
1284:         draw.line((60, 105, 1540, 105), fill="#B7D7EA", width=3)
1285:
1286:         boxes: dict[str, tuple[int, int, int, int]] = {}
1287:         for node_id, label, x, y, w, h, fill in spec["nodes"]:
1288:             boxes[node_id] = (x, y, x + w, y + h)
1289:
1290:         for source_id, target_id, *maybe_label in spec["edges"]:
1291:             source = boxes[source_id]
1292:             target = boxes[target_id]
1293:             start, end = edge_points(source, target)
1294:             draw_arrow(draw, start, end, "#2563EB", width=4)
1295:             if maybe_label:
1296:                 label = maybe_label[0]
1297:                 mid = ((start[0] + end[0]) // 2, (start[1] + end[1]) // 2)
1298:                 text_box = label_font.getbbox(label)
1299:                 tw = text_box[2] - text_box[0] + 16
1300:                 th = text_box[3] - text_box[1] + 10
1301:                 if math.hypot(end[0] - start[0], end[1] - start[1]) > max(tw + 28, 95):
1302:                     draw.rounded_rectangle((mid[0] - tw // 2, mid[1] - th // 2, mid[0] + tw // 2, mid[1]
+ th // 2), radius=9, fill="#EFF6FF", outline="#BFDBFE", width=1)
1303:                     draw.text((mid[0] - tw // 2 + 8, mid[1] - th // 2 + 4), label, fill="#1E3A8A",
font=label_font)
1304:
1305:         for node_id, label, x, y, w, h, fill in spec["nodes"]:
1306:             draw.rounded_rectangle((x, y, x + w, y + h), radius=20, fill=fill, outline="#668EA8",
width=3)
1307:             lines = wrap_for_box(label, box_font, w - 28)
1308:             line_height = 30
1309:             total_h = len(lines) * line_height
1310:             start_y = y + (h - total_h) // 2
1311:             for index, line in enumerate(lines):
1312:                 bbox = box_font.getbbox(line)
1313:                 tw = bbox[2] - bbox[0]
1314:                 draw.text((x + (w - tw) // 2, start_y + index * line_height), line, fill="#102033",
font=box_font)
1315:
1316:         path = DIAGRAM_DIR / f"{spec['name']}.png"
1317:         image.save(path)
1318:         diagram_paths[spec["name"]] = path
1319:     return diagram_paths
1320:
1321:
```

add_diagram (docs/build_complete_guide.py, lines 1322-1335)

add_diagram named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1322-1335 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, add_paragraph, add_run, add_picture, str, Inches, Pt, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1324: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1322: def add_diagram(doc: Document, path: Path, caption: str) -> None:
1323:     if not path.exists():
1324:         return
1325:     paragraph = doc.add_paragraph()
1326:     paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
1327:     run = paragraph.add_run()
1328:     run.add_picture(str(path), width=Inches(6.35))
1329:     cap = doc.add_paragraph(caption)
1330:     cap.alignment = WD_ALIGN_PARAGRAPH.CENTER
1331:     for run in cap.runs:
1332:         run.font.size = Pt(9)
1333:         run.font.color.rgb = RGBColor(85, 85, 85)
1334:
1335:
```

shade (docs/build_complete_guide.py, lines 1336-1342)

shade named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1336-1342 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get_or_add_tcPr, OxmlElement, set, qn, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1336: def shade(cell, fill: str) -> None:
1337:     tc_pr = cell._tc.get_or_add_tcPr()
1338:     shd = OxmlElement("w:shd")
1339:     shd.set(qn("w:fill"), fill)
1340:     tc_pr.append(shd)
1341:
1342:
```

cell_text (docs/build_complete_guide.py, lines 1343-1351)

cell_text named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1343-1351 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_run, str, and Pt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1343: def cell_text(cell, text: str, bold: bool = False) -> None:
1344:     cell.text = ""
1345:     run = cell.paragraphs[0].add_run(str(text))
1346:     run.bold = bold
1347:     run.font.name = "Calibri"
1348:     run.font.size = Pt(9)
1349:     cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
1350:
1351:
```

add_table (docs/build_complete_guide.py, lines 1352-1368)

add_table named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1352-1368 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Inches, cell_text, shade, and add_row. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1352: def add_table(doc: Document, rows: list[tuple[str, str]]) -> None:
1353:     table = doc.add_table(rows=1, cols=2)
1354:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1355:     table.autofit = False
1356:     table.columns[0].width = Inches(1.55)
1357:     table.columns[1].width = Inches(4.95)
1358:     head = table.rows[0].cells
1359:     cell_text(head[0], "Item", True)
1360:     cell_text(head[1], "Explanation", True)
1361:     shade(head[0], "E8EEF5")
1362:     shade(head[1], "E8EEF5")
1363:     for label, value in rows:
1364:         cells = table.add_row().cells
1365:         cell_text(cells[0], label, True)
1366:         cell_text(cells[1], value, False)
1367:
1368:
```

add_code (docs/build_complete_guide.py, lines 1369-1376)

add_code named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1369-1376 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph, add_run, rstrip, set, and qn. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1369: def add_code(doc: Document, text: str) -> None:
1370:     p = doc.add_paragraph()
1371:     p.style = doc.styles["CodeBlock"]
1372:     run = p.add_run((text or "").rstrip())
1373:     run.font.name = "Consolas"
1374:     run._element.rPr.rFonts.set(qn("w:eastAsia"), "Consolas")
1375:
1376:
```

add_note (docs/build_complete_guide.py, lines 1377-1391)

add_note named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1377-1391 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_table, Inches, cell, shade, add_run, RGBColor, Pt, and add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1377: def add_note(doc: Document, title: str, body: str) -> None:
1378:     table = doc.add_table(rows=1, cols=1)
1379:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1380:     table.autofit = False
1381:     table.columns[0].width = Inches(6.35)
1382:     cell = table.cell(0, 0)
1383:     shade(cell, "F4F8FC")
1384:     title_run = cell.paragraphs[0].add_run(title)
1385:     title_run.bold = True
```

```
1386:     title_run.font.color.rgb = RGBColor(31, 77, 120)
1387:     title_run.font.size = Pt(10)
1388:     paragraph = cell.add_paragraph(body)
1389:     paragraph.style = "Body Text"
1390:
1391:
```

bullets (docs/build_complete_guide.py, lines 1392-1396)

bullets named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1392-1396 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1392: def bullets(doc: Document, items: list[str]) -> None:
1393:     for item in items:
1394:         doc.add_paragraph(item, style="List Bullet")
1395:
1396:
```

numbers (docs/build_complete_guide.py, lines 1397-1401)

numbers named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1397-1401 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1397: def numbers(doc: Document, items: list[str]) -> None:
1398:     for item in items:
1399:         doc.add_paragraph(item, style="List Number")
1400:
1401:
```

setup_document (docs/build_complete_guide.py, lines 1402-1451)

setup_document named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1402-1451 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Document, Inches, set, qn, Pt, from_string, add_style, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1449: return doc.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1402: def setup_document() -> Document:
1403:     doc = Document()
1404:     section = doc.sections[0]
1405:     section.page_height = Inches(11)
1406:     section.page_width = Inches(8.5)
1407:     for section in doc.sections:
1408:         section.top_margin = Inches(0.65)
1409:         section.bottom_margin = Inches(0.6)
1410:         section.left_margin = Inches(0.65)
1411:         section.right_margin = Inches(0.65)
```

```
        section.header_distance = Inches(0.28)
        section.footer_distance = Inches(0.28)

    normal = doc.styles["Normal"]
    normal.font.name = "Calibri"
    normal._element.rPr.rFonts.set(qn("w:eastAsia"), "Calibri")
    normal.font.size = Pt(10)
    normal.paragraph_format.space_after = Pt(3)
    normal.paragraph_format.line_spacing = 1.08

    for name, size, color, before, after in [
        ("Heading 1", 14, "2E74B5", 9, 4),
        ("Heading 2", 12, "2E74B5", 7, 3),
        ("Heading 3", 10.5, "1F4D78", 5, 2),
    ]:
        style = doc.styles[name]
        style.font.name = "Calibri"
        style._element.rPr.rFonts.set(qn("w:eastAsia"), "Calibri")
        style.font.size = Pt(size)
        style.font.color.rgb = RGBColor.from_string(color)
        style.paragraph_format.space_before = Pt(before)
        style.paragraph_format.space_after = Pt(after)

    code = doc.styles.add_style("CodeBlock", 1)
    code.font.name = "Consolas"
    code._element.rPr.rFonts.set(qn("w:eastAsia"), "Consolas")
    code.font.size = Pt(7.2)
    code.paragraph_format.space_before = Pt(2)
    code.paragraph_format.space_after = Pt(3)
    code.paragraph_format.line_spacing = 1.0

    header = doc.sections[0].header.paragraphs[0]
    header.text = "OMB Portfolio Builder and Website Complete Guide"
    header.alignment = WD_ALIGN_PARAGRAPH.RIGHT
    for run in header.runs:
        run.font.size = Pt(8)
        run.font.color.rgb = RGBColor(85, 85, 85)
    return doc


```

add_cover (docs/build_complete_guide.py, lines 1452-1469)

add_cover named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1452-1469 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph, add_run, Pt, RGBColor, get, add_note, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def add_cover(doc: Document, package: dict) -> None:
    title = doc.add_paragraph()
    title.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = title.add_run("OMB Portfolio Builder and Website\nComplete Creation Guide")
    run.bold = True
    run.font.size = Pt(26)
    run.font.color.rgb = RGBColor(11, 37, 69)
    subtitle = doc.add_paragraph()
    subtitle.alignment = WD_ALIGN_PARAGRAPH.CENTER
    r = subtitle.add_run("A beginner-friendly manual explaining the desktop app, public website, GitHub
workflows, installer, parser, compiler tools, AI backend, and publishing system.")
    r.font.size = Pt(12)
    meta = doc.add_paragraph()
    meta.alignment = WD_ALIGN_PARAGRAPH.CENTER
    meta.add_run(f"Repository: otienomaurice/omb-portfolio-builder\nVersion documented:
{package.get('version', 'unknown')}\nGenerated: July 4, 2026")
    add_note(doc, "How this guide is written", "This guide starts from the highest-level idea and then
drills down. It intentionally explains terms that engineers usually skip: what Electron is, what a workflow is,
what a local backend is, why Git branches exist, and why the builder and website are separated.")
    doc.add_page_break()


```

add_reading_map (docs/build_complete_guide.py, lines 1470-1483)

add_reading_map named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1470-1483 and is part of the documentation and generated reference area. used to explain the app and source

structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1470: def add_reading_map(doc: Document) -> None:
1471:     doc.add_heading("Reading Map", level=1)
1472:     doc.add_paragraph("Use this manual in layers. First read the high-level design pages. Then read the
app flow pages. After that, use the file-by-file and command pages as a reference when you need to understand a
specific file or command.")
1473:     add_table(doc, [
1474:         ("Part 1", "High-level architecture and big-picture mental model."),
1475:         ("Part 2", "Builder app internals: Electron, local backend, parser, rich editors, previews,
compile code, and publishing."),
1476:         ("Part 3", "Public website internals: HTML, CSS, JavaScript, projects.json, assets, search, AI,
and mobile behavior."),
1477:         ("Part 4", "GitHub, branches, workflows, releases, installers, and updates."),
1478:         ("Part 5", "Every tracked file explained in plain English."),
1479:         ("Part 6", "Commands, tradeoffs, troubleshooting, and safe operating procedures."),
1480:     ])
1481:     doc.add_page_break()
1482:
1483:
```

add_entry_points (docs/build_complete_guide.py, lines 1484-1500)

add_entry_points named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1484-1500 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, numbers, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1484: def add_entry_points(doc: Document) -> None:
1485:     doc.add_heading("Where Someone Starts: Entry Points", level=1)
1486:     doc.add_paragraph("An entry point is the first file, executable, or function that starts a flow. This
 project has several entry points because normal users, developers, GitHub Actions, Cloudflare, and installers
all start in different places.")
1487:     add_table(doc, [(name, f"{entry}. {meaning}") for name, entry, meaning in ENTRY_POINTS])
1488:     doc.add_heading("Fastest way to orient yourself", level=2)
1489:     numbers(doc, [
1490:         "If you are using the installed app, start with OMB Portfolio Builder.exe.",
1491:         "If you are debugging desktop startup, start with package.json and main.cjs.",
1492:         "If you are debugging local APIs, start with server.mjs.",
1493:         "If you are debugging builder UI, start with template-preview.html, template-preview.js, and
template-preview.css.",
1494:         "If you are debugging the public website, start with index.html, script.js, styles.css, and
projects.json.",
1495:         "If you are debugging installer behavior, start with package.json build.nsis and
build/installer.nsh.",
1496:         "If you are debugging releases, start with .github/workflows/build-windows-builder.yml.",
1497:     ])
1498:     doc.add_page_break()
1499:
1500:
```

add_flow_walkthroughs (docs/build_complete_guide.py, lines 1501-1511)

add_flow_walkthroughs named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1501-1511 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1501: def add_flow_walkthroughs(doc: Document, diagrams: dict[str, Path]) -> None:
 1502:     for title, diagram_name, steps, debug_note in FLOW_WALKTHROUGHS:
 1503:         doc.add_heading(f"Detailed Walkthrough: {title}", level=1)
 1504:         add_diagram(doc, diagrams[diagram_name], f"Context diagram for {title}.")
 1505:         doc.add_paragraph("This walkthrough follows the real direction of work through the system. Read
it slowly: each line is one handoff from one layer to another.")
 1506:         add_table(doc, steps)
 1507:         doc.add_heading("Debug note", level=2)
 1508:         doc.add_paragraph(debug_note)
 1509:         doc.add_page_break()
 1510:
 1511:
```

add_data_contracts (docs/build_complete_guide.py, lines 1512-1524)

add_data_contracts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1512-1524 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_code, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1512: def add_data_contracts(doc: Document) -> None:
 1513:     for title, purpose, example, note in DATA_CONTRACTS:
 1514:         doc.add_heading(f"Data Contract: {title}", level=1)
 1515:         doc.add_paragraph(purpose)
 1516:         doc.add_heading("Example shape", level=2)
 1517:         add_code(doc, example)
 1518:         doc.add_heading("How to read this", level=2)
 1519:         doc.add_paragraph(note)
 1520:         doc.add_heading("Why contracts matter", level=2)
 1521:         doc.add_paragraph("A data contract is an agreement between two pieces of software. If the
frontend sends one shape and the backend expects another shape, the feature breaks even if both files are
individually valid code.")
 1522:         doc.add_page_break()
 1523:
 1524:
```

endpoint_purpose (docs/build_complete_guide.py, lines 1525-1554)

endpoint_purpose named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1525-1554 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1527: return "Compiles or runs a source file and returns terminal output."; line 1529: return "Saves source code into the local compile workspace."; line 1531: return "Formats source code for cleaner display and editing."; line 1533: return "Checks compiler/runtime availability.". There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1525: def endpoint_purpose(endpoint: str) -> str:
 1526:     if "code/compile" in endpoint:
 1527:         return "Compiles or runs a source file and returns terminal output."
 1528:     if "code/save" in endpoint:
 1529:         return "Saves source code into the local compile workspace."
 1530:     if "code/beautify" in endpoint:
```

```
        return "Formats source code for cleaner display and editing."
    if "code/tools" in endpoint:
        return "Checks compiler/runtime availability."
    if "code/install" in endpoint:
        return "Attempts to install missing compiler tools."
    if "publish-target" in endpoint:
        return "Reads or writes the Git publishing target and authorization state."
    if "publish" in endpoint:
        return "Publishes generated website files to the target repository."
    if "catalog" in endpoint:
        return "Loads project/catalog data for the builder."
    if "templates" in endpoint:
        return "Loads appearance template definitions."
    if "app-update" in endpoint:
        return "Checks or starts builder app update behavior."
    if "security-report" in endpoint:
        return "Returns security/download/auth reporting for the builder."
    if "portfolio-ai" in endpoint:
        return "Handles AI assistant questions."
    if "system-check" in endpoint:
        return "Checks local tool/system readiness."
    return "Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact
behavior."


```

extract_api_endpoints (docs/build_complete_guide.py, lines 1555-1567)

extract_api_endpoints named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1555-1567 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, enumerate, read_text, splitlines, findall, rstrip, min, get, str, sorted, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1565: return [(endpoint, file_name, str(line_no)) for (endpoint, file_name), line_no in sorted(endpoints.items())].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def extract_api_endpoints() -> list[tuple[str, str, str]]:
    endpoints: dict[tuple[str, str], int] = {}
    for file_name in ["template-preview.js", "script.js", "server.mjs", "index.html"]:
        path = REPO / file_name
        if not path.exists():
            continue
        for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
start=1):
            for match in re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", line):
                clean = match.rstrip("/")
                endpoints[(clean, file_name)] = min(line_no, endpoints.get((clean, file_name), line_no))
    return [(endpoint, file_name, str(line_no)) for (endpoint, file_name), line_no in
sorted(endpoints.items())]


```

add_api_endpoint_catalog (docs/build_complete_guide.py, lines 1568-1590)

add_api_endpoint_catalog named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1568-1590 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references extract_api_endpoints, add_heading, add_paragraph, add_page_break, endpoint_purpose, and add_table. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1575: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def add_api_endpoint_catalog(doc: Document) -> None:
    endpoints = extract_api_endpoints()
```

```
 1570:     doc.add_heading("API Endpoint Catalog", level=1)
 1571:     doc.add_paragraph("This catalog is generated from strings found in the source files. It tells you
which /api paths are used and where to start reading when a request fails.")
 1572:     if not endpoints:
 1573:         doc.add_paragraph("No /api endpoints were found by the scanner.")
 1574:         doc.add_page_break()
 1575:         return
 1576:     rows = [(endpoint, f"Seen in {file_name} near line {line_no}. Purpose: {endpoint_purpose(endpoint)}")
 for endpoint, file_name, line_no in endpoints]
 1577:     add_table(doc, rows)
 1578:     doc.add_page_break()
 1579:     for endpoint, file_name, line_no in endpoints:
 1580:         doc.add_heading(f"API Detail: {endpoint}", level=1)
 1581:         add_table(doc, [
 1582:             ("Where seen", f"{file_name} near line {line_no}"),
 1583:             ("Purpose", endpoint_purpose(endpoint)),
 1584:             ("Frontend question", "Which button, form, or page action sends this request?"),
 1585:             ("Backend question", "Which handler in server.mjs receives this path and what files/tools
does it touch?"),
 1586:             ("Debugging", "Check browser network status, response JSON, server logs, and whether stale
cache was avoided with no-store or a timestamp."),
 1587:         ])
 1588:         doc.add_page_break()
 1589:
 1590:
```

function_purpose (docs/build_complete_guide.py, lines 1591-1629)

function_purpose named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1591-1629 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1594: return "Renders UI, markup, or display output."; line 1596: return "Handles an event, request, command, or user action."; line 1598: return "Updates UI, state, cache, or stored data."; line 1600: return "Persists data to local state, disk, credentials, or browser storage.". There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1591: def function_purpose(file_name: str, function_name: str) -> str:
 1592:     name = function_name.lower()
 1593:     if name.startswith("render"):
 1594:         return "Renders UI, markup, or display output."
 1595:     if name.startswith("handle"):
 1596:         return "Handles an event, request, command, or user action."
 1597:     if name.startswith("update"):
 1598:         return "Updates UI, state, cache, or stored data."
 1599:     if name.startswith("save") or name.startswith("write") or name.startswith("store"):
 1600:         return "Persists data to local state, disk, credentials, or browser storage."
 1601:     if name.startswith("read") or name.startswith("load") or name.startswith("fetch"):
 1602:         return "Loads data from disk, network, browser storage, or a service."
 1603:     if "compile" in name or "compiler" in name:
 1604:         return "Part of the Compile Code language/tool/build/run flow."
 1605:     if "publish" in name or "git" in name or "remote" in name:
 1606:         return "Part of publishing, Git, target repository, or authorization behavior."
 1607:     if "auth" in name or "credential" in name or "access" in name:
 1608:         return "Part of authentication, authorization, or credential checking."
 1609:     if "cache" in name:
 1610:         return "Reads, writes, or validates cached state."
 1611:     if name.startswith("validate") or name.startswith("normalize") or name.startswith("safe") or
name.startswith("sanitize"):
 1612:         return "Validates or normalizes input so later code receives safe predictable values."
 1613:     if name.startswith("create") or name.startswith("new") or name.startswith("build"):
 1614:         return "Creates an object, UI structure, process, payload, or generated artifact."
 1615:     if name.startswith("open") or name.startswith("close") or name.startswith("show") or
name.startswith("hide"):
 1616:         return "Controls a window, dialog, panel, menu, or visible state."
 1617:     if name.startswith("sync"):
 1618:         return "Keeps two places aligned, such as local data and target files."
 1619:     if file_name == "main.cjs":
 1620:         return "Part of Electron desktop startup, window control, workspace setup, menu behavior, or
update handoff."
 1621:     if file_name == "server.mjs":
 1622:         return "Part of the local backend API, filesystem, compiler, Git, AI, update, or publish
workflow."
 1623:     if file_name == "template-preview.js":
 1624:         return "Part of the private builder frontend and editing experience."
 1625:     if file_name == "script.js":
 1626:         return "Part of the public website runtime."
 1627:     return "Named function in the repository. Inspect the surrounding lines to learn its exact inputs and
 outputs."
```

```
1628:
1629:
```

line_excerpt (docs/build_complete_guide.py, lines 1664-1673)

line_excerpt named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1664-1673 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, len, enumerate, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1671: return "\n".join(rendered).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1664: def line_excerpt(lines: list[str], start: int, end: int, max_lines: int = 120) -> str:
1665:     selected = lines[max(start - 1, 0): min(end, len(lines))]
1666:     truncated = len(selected) > max_lines
1667:     selected = selected[:max_lines]
1668:     rendered = [f"{start + index:>5}: {line}" for index, line in enumerate(selected)]
1669:     if truncated:
1670:         rendered.append(f"      ... excerpt truncated after {max_lines} lines. Open the source file for
the rest of this function.")
1671:     return "\n".join(rendered)
1672:
1673:
```

find_block_end (docs/build_complete_guide.py, lines 1674-1689)

find_block_end named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1674-1689 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references range, min, len, sub, count, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1684: return index + 1; line 1686: return index; line 1687: return min(start_index + 80, len(lines)).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1674: def find_block_end(lines: list[str], start_index: int) -> int:
1675:     brace_balance = 0
1676:     saw_brace = False
1677:     for index in range(start_index, min(start_index + 500, len(lines))):
1678:         line = re.sub(r"//.*$", "", lines[index])
1679:         brace_balance += line.count("{")
1680:         brace_balance -= line.count("}")
1681:         if "{" in line:
1682:             saw_brace = True
1683:         if saw_brace and brace_balance <= 0 and index > start_index:
1684:             return index + 1
1685:         if not saw_brace and index > start_index and not lines[index].startswith((" ", "\t")):
1686:             return index
1687:     return min(start_index + 80, len(lines))
1688:
1689:
```

extract_function_blocks (docs/build_complete_guide.py, lines 1690-1750)

This function scans source files, finds function declarations, estimates function boundaries, and records calls, returns, storage keys, endpoints, and source excerpts.

The documentation generator depends on it to build source-aware function explanations instead of hand-maintained stale lists.

It calls or references enumerate, strip, search, group, lstrip, startswith, next, range, len, min, and 14 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1748: return functions.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1690: def extract_function_blocks(repo_file: str, lines: list[str]) -> list[dict]:
1691:     functions = []
1692:     for index, line in enumerate(lines):
1693:         stripped = line.strip()
1694:         for pattern in FUNCTION_PATTERNS:
1695:             match = pattern.search(line)
1696:             if not match:
1697:                 continue
1698:             name = match.group(1)
1699:             if line.lstrip().startswith("Function "):
1700:                 end = next((i + 1 for i in range(index + 1, len(lines)) if
lines[i].lstrip().startswith("FunctionEnd")), min(index + 80, len(lines)))
1701:             elif line.lstrip().startswith("def "):
1702:                 end = next((i for i in range(index + 1, len(lines)) if lines[i] and not
lines[i].startswith((" ", "\t", "@"))), min(index + 120, len(lines)))
1703:             else:
1704:                 end = find_block_end(lines, index)
1705:             body = "\n".join(lines[index:end])
1706:             call_candidates = re.findall(r"\b([A-Za-z_$][\w$]*)\s*\(", body)
1707:             calls = []
1708:             for candidate in call_candidates:
1709:                 if candidate not in COMMON_CALL_WORDS and candidate != name and candidate not in calls:
1710:                     calls.append(candidate)
1711:             endpoint_hits = sorted(set(match.rstrip("/") for match in
re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", body)))
1712:             local_storage_keys = sorted(set(match for match in
re.findall(r"(?:localStorage|sessionStorage)\.(?:getItem|setItem|removeItem)\(['\"]([^'\"]+)", body)))
1713:             return_lines = []
1714:             local_variables = []
1715:             for offset, body_line in enumerate(lines[index:end], start=index + 1):
1716:                 stripped_body_line = body_line.strip()
1717:                 if re.search(r"\breturn\b", stripped_body_line):
1718:                     return_lines.append({"line": offset, "statement": stripped_body_line[:220]})
1719:                 var_match = re.match(r"^\s*(?:const|let|var)\s+([A-Za-z_$][\w$]*)\s*=", body_line)
1720:                 if var_match and len(local_variables) < 18:
1721:                     local_variables.append({
1722:                         "line": offset,
1723:                         "name": var_match.group(1),
1724:                         "statement": stripped_body_line[:220],
1725:                     })
1726:             functions.append({
1727:                 "line": index + 1,
1728:                 "end_line": end,
1729:                 "name": name,
1730:                 "purpose": function_purpose(Path(repo_file).name, name),
1731:                 "signature": stripped[:240],
1732:                 "excerpt": line_excerpt(lines, index + 1, end, max_lines=120),
1733:                 "line_count": max(end - index, 1),
1734:                 "calls": calls[:30],
1735:                 "endpoints": endpoint_hits[:30],
1736:                 "storage_keys": local_storage_keys[:30],
1737:                 "returns": len(re.findall(r"\breturn\b", body)),
1738:                 "return_lines": return_lines[:12],
1739:                 "local_variables": local_variables,
1740:                 "awaits": len(re.findall(r"\bawait\b", body)),
1741:                 "touches_dom":
bool(re.search(r"document\.|querySelector|getElementById|classList|dataset", body)),
1742:                 "touches_files":
bool(re.search(r"readFile|writeFile|appendFile|copyFile|mkdir|rm|unlink|fs\.", body, re.IGNORECASE)),
1743:                 "runs_process": bool(re.search(r"\bspawn\b|\bexecFile\b|\bexec\b", body, re.IGNORECASE)),
1744:                 "uses_network": bool(re.search(r"\bfetch\s*\(|https?://|/api/", body, re.IGNORECASE)),
1745:                 "uses_security":
bool(re.search(r"auth|token|secret|credential|permission|authorize|scope", body, re.IGNORECASE)),
1746:             })
1747:             break
1748:     return functions
1749:
1750:
```

collect_signal_hits (docs/build_complete_guide.py, lines 1751-1768)

collect_signal_hits named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1751-1768 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references enumerate, strip, search, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1766: return hits.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1751: def collect_signal_hits(lines: list[str]) -> list[dict]:
1752:     hits = []
1753:     for line_no, line in enumerate(lines, start=1):
1754:         stripped = line.strip()
1755:         if not stripped:
1756:             continue
1757:         for name, pattern, explanation in SOURCE_SIGNAL_RULES:
1758:             if pattern.search(line):
1759:                 hits.append({
1760:                     "line": line_no,
1761:                     "type": name,
1762:                     "evidence": stripped[:220],
1763:                     "meaning": explanation,
1764:                 })
1765:                 break
1766:     return hits
1767:
1768:
```

function_detail_rows (docs/build_complete_guide.py, lines 1769-1793)

function_detail_rows named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1769-1793 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, join, expression, and statement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1781: return [; line 1789: ("Async/return clues", f"{item['awaits']} await expression(s), {item['returns']} return statement(s)."),.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1769: def function_detail_rows(item: dict) -> list[tuple[str, str]]:
1770:     touches = []
1771:     if item["touches_dom"]:
1772:         touches.append("browser/Electron DOM")
1773:     if item["touches_files"]:
1774:         touches.append("filesystem")
1775:     if item["runs_process"]:
1776:         touches.append("external processes")
1777:     if item["uses_network"]:
1778:         touches.append("network/API requests")
1779:     if item["uses_security"]:
1780:         touches.append("authentication/security data")
1781:     return [
1782:         ("Source location", f"Lines {item['line']}-{item['end_line']} ({item['line_count']} source
lines)."),
1783:         ("Purpose", item["purpose"]),
1784:         ("Declaration", item["signature"]),
1785:         ("Touches", ", ".join(touches) if touches else "Mostly local calculations or local UI state."),
1786:         ("Calls detected", ", ".join(item["calls"][:12]) if item["calls"] else "No obvious named calls
detected by the generator."),
1787:         ("API endpoints", ", ".join(item["endpoints"]) if item["endpoints"] else "None detected inside
this function body."),
1788:         ("Storage keys", ", ".join(item["storage_keys"]) if item["storage_keys"] else "No local/session
storage keys detected."),
1789:         ("Async/return clues", f"{item['awaits']} await expression(s), {item['returns']} return
statement(s)."),
1790:         ("Debug approach", "Trigger the user action that reaches this function, inspect inputs/state
before the function, then inspect the returned value, DOM update, file write, process output, or API response
after it runs."),
1791:     ]
1792:
1793:
```

function_detail (docs/build_complete_guide.py, lines 1794-1801)

function_detail named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1794-1801 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, lower, and folder_role. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1796: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1794: def function_detail(item: dict, repo_file: str) -> dict:
1795:     custom = IMPORTANT_FUNCTION_DETAILS.get((repo_file, item["name"]), {})
1796:     return {
1797:         "what": custom.get("what") or f"{item['name']} {item['purpose'].lower()} In this file, it appears
around lines {item['line']}-{item['end_line']} and is part of the {folder_role(repo_file).lower()}",
1798:         "why": custom.get("why") or f"This function is needed so {repo_file} can keep that responsibility
in one named place instead of scattering it through unrelated event handlers or route code.",
1799:     }
1800:
1801:
```

sentence_list (docs/build_complete_guide.py, lines 1802-1812)

sentence_list named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1802-1812 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references str, len, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1804: return empty; line 1809: return selected[0]; line 1810: return ", ".join(selected[:-1]) + f", and {selected[-1]}".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1802: def sentence_list(values: list[str], empty: str, limit: int = 10) -> str:
1803:     if not values:
1804:         return empty
1805:     selected = [str(value) for value in values[:limit] if str(value)]
1806:     if len(values) > limit:
1807:         selected.append(f"{len(values) - limit} more")
1808:     if len(selected) == 1:
1809:         return selected[0]
1810:     return ", ".join(selected[:-1]) + f", and {selected[-1]}"
1811:
1812:
```

function_calls_paragraph (docs/build_complete_guide.py, lines 1813-1819)

function_calls_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1813-1819 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sentence_list, and get. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1817: return f"It calls or references {calls}. Endpoint references found inside it are {endpoints}. Browser storage keys found inside it are {storage}.".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1813: def function_calls_paragraph(item: dict) -> str:
1814:     calls = sentence_list(item.get("calls", []), "no major named helper calls detected")
1815:     endpoints = sentence_list(item.get("endpoints", []), "no direct /api endpoint strings detected")
1816:     storage = sentence_list(item.get("storage_keys", []), "no local/session storage keys detected")
1817:     return f"It calls or references {calls}. Endpoint references found inside it are {endpoints}. Browser
storage keys found inside it are {storage}."
1818:
1819:
```

function_returns_paragraph (docs/build_complete_guide.py, lines 1820-1834)

function_returns_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1820-1834 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, join, len, and statement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1824: extra = "" if len(return_lines) <= 4 else f" There are {len(return_lines) - 4} additional return statements in the function body."; line 1825: return f"It has {len(return_lines)} return statement(s). The most important visible returns are: {snippets}.{extra}"; line 1827: return "It does not expose many explicit return statements, but it produces its result through process output, side effects, or a structured response built after external commands finish."; line 1829: return "It does not mainly return a value; its output is the changed screen state or DOM it updates.". There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1820: def function_returns_paragraph(item: dict) -> str:
 1821:     return_lines = item.get("return_lines", [])
 1822:     if return_lines:
 1823:         snippets = "; ".join(f"line {entry['line']}: {entry['statement']}" for entry in return_lines[:4])
 1824:         extra = "" if len(return_lines) <= 4 else f" There are {len(return_lines) - 4} additional return
statements in the function body."
 1825:         return f"It has {len(return_lines)} return statement(s). The most important visible returns are:
{snippets}.{extra}"
 1826:     if item.get("runs_process"):
 1827:         return "It does not expose many explicit return statements, but it produces its result through
process output, side effects, or a structured response built after external commands finish."
 1828:     if item.get("touches_dom"):
 1829:         return "It does not mainly return a value; its output is the changed screen state or DOM it
updates."
 1830:     if item.get("touches_files"):
 1831:         return "Its result is mostly side effect based: it reads, writes, copies, or synchronizes files
rather than returning a simple value."
 1832:     return "It has no obvious return statement in the extracted body; it likely completes through state
changes, helper calls, or event flow."
 1833:
 1834:
```

function_variables_paragraph (docs/build_complete_guide.py, lines 1835-1843)

function_variables_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1835-1843 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, join, len, and variable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1838: return "No major local variable declarations were detected in the extracted function body."; line 1841: return f"Important local values include {snippets}{extra}. These variables show what the function is assembling, checking, or handing to another layer.".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1835: def function_variables_paragraph(item: dict) -> str:
 1836:     variables = item.get("local_variables", [])
 1837:     if not variables:
 1838:         return "No major local variable declarations were detected in the extracted function body."
 1839:     snippets = "; ".join(f"{entry['name']} at line {entry['line']}" for entry in variables[:8])
 1840:     extra = "" if len(variables) <= 8 else f"; plus {len(variables) - 8} more local variable(s)"
 1841:     return f"Important local values include {snippets}{extra}. These variables show what the function is
assembling, checking, or handing to another layer."
 1842:
 1843:
```

add_function_explanation_docx (docs/build_complete_guide.py, lines 1844-1856)

add_function_explanation_docx named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1844-1856 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references function_detail, add_heading, add_paragraph, function_calls_paragraph, function_returns_paragraph, function_variables_paragraph, and add_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1844: def add_function_explanation_docx(doc: Document, repo_file: str, item: dict, include_excerpt: bool =
True, heading_level: int = 3) -> None:
 1845:     details = function_detail(item, repo_file)
 1846:     doc.add_heading(f"{item['name']} ({repo_file}, lines {item['line']}-{item['end_line']})",
level=heading_level)
 1847:     doc.add_paragraph(details["what"])
 1848:     doc.add_paragraph(details["why"])
 1849:     doc.add_paragraph(function_calls_paragraph(item))
 1850:     doc.add_paragraph(function_returns_paragraph(item))
 1851:     doc.add_paragraph(function_variables_paragraph(item))
 1852:     if include_excerpt:
 1853:         doc.add_paragraph("Relevant source excerpt:")
 1854:         add_code(doc, item["excerpt"])
 1855:
 1856:
```

add_function_explanation_pdf (docs/build_complete_guide.py, lines 1857-1869)

add_function_explanation_pdf named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1857-1869 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references function_detail, append, Paragraph, pdf_paragraph, function_calls_paragraph, function_returns_paragraph, function_variables_paragraph, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1857: def add_function_explanation_pdf(story: list, repo_file: str, item: dict, styles, include_excerpt: bool =
 True) -> None:
 1858:     details = function_detail(item, repo_file)
 1859:     story.append(Paragraph(f"{item['name']} ({repo_file}, lines {item['line']}-{item['end_line']})",
styles["Heading3"]))
 1860:     story.append(pdf_paragraph(details["what"], styles["SmallBody"]))
 1861:     story.append(pdf_paragraph(details["why"], styles["SmallBody"]))
 1862:     story.append(pdf_paragraph(function_calls_paragraph(item), styles["SmallBody"]))
 1863:     story.append(pdf_paragraph(function_returns_paragraph(item), styles["SmallBody"]))
 1864:     story.append(pdf_paragraph(function_variables_paragraph(item), styles["SmallBody"]))
 1865:     if include_excerpt:
 1866:         story.append(pdf_paragraph("Relevant source excerpt:", styles["SmallBody"]))
 1867:         story.append(pdf_code(item["excerpt"], styles))
 1868:
 1869:
```

important_function_blocks (docs/build_complete_guide.py, lines 1870-1886)

important_function_blocks named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1870-1886 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references extract_source_facts, sorted, and, exists, get, next, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1884: return selected.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1870: def important_function_blocks(files: list[str]) -> list[tuple[str, dict]]:
 1871:     facts_by_file = {
```

```
1872:         repo_file: extract_source_facts(repo_file)
1873:         for repo_file in sorted({repo_file for repo_file, _name in IMPORTANT_FUNCTIONS})
1874:         if repo_file in files and (REPO / repo_file).exists()
1875:     }
1876:     selected = []
1877:     for repo_file, function_name in IMPORTANT_FUNCTIONS:
1878:         facts = facts_by_file.get(repo_file)
1879:         if not facts:
1880:             continue
1881:         match = next((item for item in facts["functions"] if item["name"] == function_name), None)
1882:         if match:
1883:             selected.append((repo_file, match))
1884:     return selected
1885:
1886:
```

extract_functions (docs/build_complete_guide.py, lines 1887-1902)

extract_functions named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1887-1902 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, enumerate, read_text, splitlines, search, group, append, and function_purpose. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1900: return function_rows.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1887: def extract_functions() -> list[tuple[str, int, str, str]]:
1888:     function_rows: list[tuple[str, int, str, str]] = []
1889:     for file_name in ["main.cjs", "server.mjs", "template-preview.js", "script.js",
"cloudflare/portfolio-ai-worker.js", "docs/build_complete_guide.py", "build/installer.nsh"]:
1890:         path = REPO / file_name
1891:         if not path.exists():
1892:             continue
1893:         for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
start=1):
1894:             for pattern in FUNCTION_PATTERNS:
1895:                 match = pattern.search(line)
1896:                 if match:
1897:                     name = match.group(1)
1898:                     function_rows.append((file_name, line_no, name, function_purpose(file_name, name)))
1899:                     break
1900:     return function_rows
1901:
1902:
```

is_text_code_file (docs/build_complete_guide.py, lines 1908-1916)

is_text_code_file named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1908-1916 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references is_generated_reference_file, endswith, is_file, and lower. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1911: return False; line 1913: return False; line 1914: return path.is_file() and path.suffix.lower() in TEXT_CODE_EXTENSIONS.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1908: def is_text_code_file(repo_file: str) -> bool:
1909:     path = REPO / repo_file
1910:     if is_generated_reference_file(repo_file):
1911:         return False
1912:     if repo_file.endswith((".docx", ".pdf", ".png", ".ico")):
1913:         return False
1914:     return path.is_file() and path.suffix.lower() in TEXT_CODE_EXTENSIONS
1915:
1916:
```

source_lines (docs/build_complete_guide.py, lines 1917-1923)

source_lines named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1917-1923 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, is_file, read_text, and splitlines. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1920: return []; line 1921: return path.read_text(encoding="utf-8", errors="replace").splitlines().

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1917: def source_lines(repo_file: str) -> list[str]:
1918:     path = REPO / repo_file
1919:     if not path.exists() or not path.is_file():
1920:         return []
1921:     return path.read_text(encoding="utf-8", errors="replace").splitlines()
1922:
1923:
```

extract_source_facts (docs/build_complete_guide.py, lines 1924-1983)

extract_source_facts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1924-1983 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references source_lines, join, extract_function_blocks, enumerate, strip, match, len, append, group, findall, and 12 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1960: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1924: def extract_source_facts(repo_file: str) -> dict:
1925:     lines = source_lines(repo_file)
1926:     text = "\n".join(lines)
1927:     functions = extract_function_blocks(repo_file, lines)
1928:     variables = []
1929:     imports = []
1930:     endpoints = []
1931:     selectors = []
1932:     events = []
1933:     routes = []
1934:     json_keys = []
1935:     for line_no, line in enumerate(lines, start=1):
1936:         stripped = line.strip()
1937:         var_match = re.match(r"^\s*(?:const|let|var)\s+([A-Za-z_$][\w$]*)\s*=", line)
1938:         if var_match and len(variables) < 140:
1939:             variables.append({
1940:                 "line": line_no,
1941:                 "name": var_match.group(1),
1942:                 "statement": stripped[:180],
1943:             })
1944:         if re.match(r"^\s*import\s+", line) or re.match(r"^\s*(?:const|let|var)\s+\S+\s*=\s*require\(",
line):
1945:             imports.append({"line": line_no, "statement": stripped[:220]})
1946:         for match in re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", line):
1947:             endpoints.append({"line": line_no, "endpoint": match.rstrip("/")})
1948:         route_match = re.search(r"\bapp\.(get|post|put|delete|patch)\(['\"]([^'\"]+)", line)
1949:         if route_match:
1950:             routes.append({"line": line_no, "method": route_match.group(1).upper(), "path":
route_match.group(2)})
1951:         if ".addEventListener(" in line:
1952:             events.append({"line": line_no, "statement": stripped[:180]})
1953:         json_match = re.match(r'^\s*"([^"]+)"\s*:', line)
1954:         if json_match and len(json_keys) < 120:
1955:             json_keys.append({"line": line_no, "key": json_match.group(1), "statement": stripped[:180]})
1956:         if repo_file.endswith(".css"):
1957:             selector_match = re.match(r"^\s*([.#A-Za-z0-9_\-:\[\]\(\)\s>,+.=\"']+)\s*\{", line)
1958:             if selector_match and len(selectors) < 140:
1959:                 selectors.append({"line": line_no, "selector": selector_match.group(1).strip()[:160]})
1960:     return {
1961:         "repo_file": repo_file,
1962:         "line_count": len(lines),
```

```
1963:         "size_bytes": (REPO / repo_file).stat().st_size if (REPO / repo_file).exists() else 0,
1964:         "description": FILE_DESCRIPTIONS.get(repo_file, "Tracked source/configuration file in the OMB
Portfolio Builder repository."),
1965:         "functions": functions,
1966:         "variables": variables,
1967:         "imports": imports,
1968:         "endpoints": endpoints,
1969:         "routes": routes,
1970:         "selectors": selectors,
1971:         "events": events,
1972:         "json_keys": json_keys,
1973:         "signals": collect_signal_hits(lines),
1974:         "mentions_server": "server.mjs" in text,
1975:         "mentions_template_preview": "template-preview" in text,
1976:         "mentions_script": "script.js" in text,
1977:         "mentions_projects_json": "projects.json" in text,
1978:         "mentions_github": "github" in text.lower(),
1979:         "mentions_cloudflare": "cloudflare" in text.lower() or "worker" in text.lower(),
1980:         "snippet": line_excerpt(lines, 1, min(72, len(lines)), max_lines=72) if lines else "",
1981:     }
1982:
1983:
```

fact_relationship_summary (docs/build_complete_guide.py, lines 1984-2000)

fact_relationship_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1984-2000 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1998: return ", ".join(related) if related else "Mostly local to its own feature area.".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1984: def fact_relationship_summary(facts: dict) -> str:
1985:     related = []
1986:     if facts.get("mentions_template_preview"):
1987:         related.append("builder frontend")
1988:     if facts.get("mentions_server"):
1989:         related.append("local backend")
1990:     if facts.get("mentions_script"):
1991:         related.append("public website runtime")
1992:     if facts.get("mentions_projects_json"):
1993:         related.append("portfolio catalog")
1994:     if facts.get("mentions_cloudflare"):
1995:         related.append("Cloudflare/AI layer")
1996:     if facts.get("mentions_github"):
1997:         related.append("GitHub/release layer")
1998:     return ", ".join(related) if related else "Mostly local to its own feature area."
1999:
2000:
```

folder_role (docs/build_complete_guide.py, lines 2001-2018)

folder_role named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2001-2018 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 2003: return "GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs."; line 2005: return "Public web metadata. Browsers, security tools, and crawlers may request this path."; line 2007: return "Public and desktop visual asset. Used for favicons, app icons, branding, or install surfaces."; line 2009: return "Packaging and installer customization. Used while creating the Windows app installer.". There are 4 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2001: def folder_role(repo_file: str) -> str:
2002:     if repo_file.startswith(".github/workflows/"):
```

```
 2003:         return "GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs."
 2004:     if repo_file.startswith(".well-known/"):
 2005:         return "Public web metadata. Browsers, security tools, and crawlers may request this path."
 2006:     if repo_file.startswith("assets/"):
 2007:         return "Public and desktop visual asset. Used for favicons, app icons, branding, or install
surfaces."
 2008:     if repo_file.startswith("build/"):
 2009:         return "Packaging and installer customization. Used while creating the Windows app installer."
 2010:     if repo_file.startswith("cloudflare/"):
 2011:         return "Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment
notes."
 2012:     if repo_file.startswith("docs/"):
 2013:         return "Documentation and generated reference area. Used to explain the app and source
structure."
 2014:     if repo_file.startswith("Backgrounds/"):
 2015:         return "Portfolio background asset folder placeholder."
 2016:     return "Repository root. Usually an entry file, app config, public website file, package manifest, or
top-level documentation."
 2017:
 2018:
```

file_category (docs/build_complete_guide.py, lines 2019-2043)

file_category named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2019-2043 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, endswith, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 11 return statement(s). The most important visible returns are: line 2022: return "Folder marker"; line 2024: return "GitHub workflow"; line 2026: return "Image/icon asset"; line 2028: return "Generated document". There are 7 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2019: def file_category(repo_file: str, path: Path) -> str:
 2020:     suffix = path.suffix.lower()
 2021:     if repo_file.endswith(".gitkeep"):
 2022:         return "Folder marker"
 2023:     if repo_file.startswith(".github/workflows/"):
 2024:         return "GitHub workflow"
 2025:     if suffix in {".png", ".ico", ".jpg", ".jpeg", ".webp", ".svg"}:
 2026:         return "Image/icon asset"
 2027:     if suffix in {".docx", ".pdf"}:
 2028:         return "Generated document"
 2029:     if suffix in {".js", ".mjs", ".cjs"}:
 2030:         return "JavaScript source"
 2031:     if suffix in {".html", ".css"}:
 2032:         return "Website/builder UI source"
 2033:     if suffix in {".json", ".toml", ".yaml", ".yml"}:
 2034:         return "Configuration or structured data"
 2035:     if suffix in {".md", ".txt", ""}:
 2036:         return "Documentation or text control file"
 2037:     if suffix == ".nsh":
 2038:         return "NSIS installer script"
 2039:     if suffix == ".py":
 2040:         return "Python tooling"
 2041:     return "Repository file"
 2042:
 2043:
```

image_metadata (docs/build_complete_guide.py, lines 2044-2059)

image_metadata named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2044-2059 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, lower, open, lstrip, upper, getattr, and str. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2046: return {}; line 2049: return {; line 2057: return {"error": str(exc)}.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2044: def image_metadata(path: Path) -> dict:
2045:     if not path.exists() or path.suffix.lower() not in {".png", ".jpg", ".jpeg", ".webp", ".ico"}:
2046:         return {}
2047:     try:
2048:         with Image.open(path) as image:
2049:             return {
2050:                 "format": image.format or path.suffix.lower().lstrip(".").upper(),
2051:                 "width": image.size[0],
2052:                 "height": image.size[1],
2053:                 "mode": image.mode,
2054:                 "frames": getattr(image, "n_frames", 1),
2055:             }
2056:     except Exception as exc:
2057:         return {"error": str(exc)}
2058:
2059:
```

file_specific_notes (docs/build_complete_guide.py, lines 2060-2088)

file_specific_notes named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2060-2088 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, startswith, and endswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2086: return notes.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2060: def file_specific_notes(repo_file: str, category: str) -> list[str]:
2061:     notes = []
2062:     if repo_file in FILE_DESCRIPTIONS:
2063:         notes.append(FILE_DESCRIPTIONS[repo_file])
2064:     if repo_file.startswith("assets/favicon"):
2065:         notes.append("This favicon participates in browser tab identity, mobile install surfaces, and
public website branding.")
2066:     if repo_file.startswith("assets/omb-app-icon"):
2067:         notes.append("This app icon is used by the Windows desktop application, installer metadata, and
shortcuts.")
2068:     if repo_file == "assets/omb-mark.png":
2069:         notes.append("This is the OMB visual mark used by the website and builder branding.")
2070:     if repo_file == "_headers":
2071:         notes.append("This file describes security and caching headers for hosts that support static
`_headers` files, such as Cloudflare Pages style deployments.")
2072:     if repo_file == ".nojekyll":
2073:         notes.append("This makes GitHub Pages serve static files directly without Jekyll filtering.")
2074:     if repo_file == ".gitattributes":
2075:         notes.append("This protects binary artifacts such as DOCX, PDF, PNG, ICO, and installers from
line-ending conversion.")
2076:     if repo_file == ".gitignore":
2077:         notes.append("This keeps temporary build output, local-only data, and generated noise out of
version control.")
2078:     if repo_file.endswith(".gitkeep"):
2079:         notes.append("This empty marker exists so Git keeps an otherwise empty folder.")
2080:     if category == "Image/icon asset":
2081:         notes.append("Do not edit binary assets with a text editor. Replace them with properly exported
image/icon files.")
2082:     if category == "GitHub workflow":
2083:         notes.append("Workflow changes should be tested on a feature branch because mistakes can block
merges or releases.")
2084:     if not notes:
2085:         notes.append("Tracked repository file used by the builder, website, installer, documentation, or
release process.")
2086:     return notes
2087:
2088:
```

extract_file_facts (docs/build_complete_guide.py, lines 2089-2118)

extract_file_facts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2089-2118 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, file_category, is_text_code_file, source_lines, exists, is_file, guess_type, str, stat, folder_role, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2116: return facts.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2089: def extract_file_facts(repo_file: str) -> dict:
2090:     path = REPO / repo_file
2091:     suffix = path.suffix.lower()
2092:     category = file_category(repo_file, path)
2093:     text_file = is_text_code_file(repo_file) or suffix in {".gitignore", ".gitattributes", ".nojekyll",
"", ".txt", ".md"}
2094:     lines = source_lines(repo_file) if text_file and path.exists() and path.is_file() else []
2095:     mime_type, _encoding = mimetypes.guess_type(str(path))
2096:     facts = {
2097:         "repo_file": repo_file,
2098:         "path": path,
2099:         "exists": path.exists(),
2100:         "size_bytes": path.stat().st_size if path.exists() and path.is_file() else 0,
2101:         "suffix": suffix or "(none)",
2102:         "category": category,
2103:         "mime_type": mime_type or "unknown",
2104:         "folder_role": folder_role(repo_file),
2105:         "description": FILE_DESCRIPTIONS.get(repo_file, "Tracked repository file used by the builder or
public website."),
2106:         "notes": file_specific_notes(repo_file, category),
2107:         "is_text": bool(lines or text_file),
2108:         "line_count": len(lines),
2109:         "snippet": line_excerpt(lines, 1, min(72, len(lines)), max_lines=72) if lines else "",
2110:         "image": image_metadata(path),
2111:     }
2112:     if is_text_code_file(repo_file):
2113:         facts["source"] = extract_source_facts(repo_file)
2114:     else:
2115:         facts["source"] = None
2116:     return facts
2117:
2118:
```

add_file_fact_summary (docs/build_complete_guide.py, lines 2119-2137)

add_file_fact_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2119-2137 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, and add_table. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2119: def add_file_fact_summary(doc: Document, facts: dict) -> None:
2120:     rows = [
2121:         ("File", facts["repo_file"]),
2122:         ("Category", facts["category"]),
2123:         ("Folder role", facts["folder_role"]),
2124:         ("Size", f"{facts['size_bytes']:,} bytes"),
2125:         ("Extension", facts["suffix"]),
2126:         ("MIME type", facts["mime_type"]),
2127:         ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text file"),
2128:     ]
2129:     if facts["image"]:
2130:         image = facts["image"]
2131:         if "error" in image:
2132:             rows.append(("Image metadata", f"Could not inspect image: {image['error']}"))
2133:         else:
2134:             rows.append(("Image metadata", f"{image['format']} {image['width']}x{image['height']}, mode
{image['mode']}, frames {image['frames']}"))
2135:     add_table(doc, rows)
2136:
2137:
```

add_file_fact_sections (docs/build_complete_guide.py, lines 2138-2163)

add_file_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2138-2163 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_source_fact_sections, add_code, and numbers. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2138: def add_file_fact_sections(doc: Document, facts: dict) -> None:
2139:     doc.add_heading("Why this file exists", level=2)
2140:     for note in facts["notes"]:
2141:         doc.add_paragraph(note, style="List Bullet")
2142:     doc.add_heading("How it is used", level=2)
2143:     doc.add_paragraph(facts["folder_role"])
2144:     if facts["source"]:
2145:         add_source_fact_sections(doc, facts["source"], include_excerpts=True)
2146:     elif facts["snippet"]:
2147:         doc.add_heading("Readable content snippet", level=2)
2148:         add_code(doc, facts["snippet"])
2149:     elif facts["image"]:
2150:         doc.add_heading("Asset handling note", level=2)
2151:         doc.add_paragraph("This file is documented by metadata instead of raw bytes. Binary image/icon
bytes are not useful to print in a Word/PDF reference. Use an image editor or icon tool when changing it.")
2152:     else:
2153:         doc.add_heading("Binary or marker note", level=2)
2154:         doc.add_paragraph("This file is either binary, empty, or a marker/config file whose value comes
from its presence and path rather than readable content.")
2155:     doc.add_heading("Safe change checklist", level=2)
2156:     numbers(doc, [
2157:         "Confirm which app, website, installer, workflow, or document consumes this file.",
2158:         "Change it on a feature branch from development.",
2159:         "Regenerate docs if the file purpose, API surface, or behavior changes.",
2160:         "Run the smallest relevant smoke test before merging into development.",
2161:     ])
2162:
2163:
```

code_reference_name (docs/build_complete_guide.py, lines 2164-2167)

code_reference_name named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2164-2167 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sub, and strip. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2165: return re.sub(r"[^A-Za-z0-9_.-]+", "__", repo_file).strip("_") + suffix.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2164: def code_reference_name(repo_file: str, suffix: str) -> str:
2165:     return re.sub(r"[^A-Za-z0-9_.-]+", "__", repo_file).strip("_") + suffix
2166:
2167:
```

remove_directory (docs/build_complete_guide.py, lines 2168-2181)

remove_directory named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2168-2181 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, remove_readonly, Path, chmod, and rmtree. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2170: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2168: def remove_directory(path: Path) -> None:
2169:     if not path.exists():
2170:         return
2171:
2172:     def remove_readonly(function, item_path, excinfo):
2173:         try:
2174:             Path(item_path).chmod(0o700)
2175:             function(item_path)
2176:         except Exception:
2177:             raise excinfo
2178:
2179:     shutil.rmtree(path, onexc=remove_readonly)
2180:
2181:
```

remove_readonly (docs/build_complete_guide.py, lines 2172-2181)

remove_readonly named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2172-2181 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Path, chmod, and rmtree. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2172:     def remove_readonly(function, item_path, excinfo):
2173:         try:
2174:             Path(item_path).chmod(0o700)
2175:             function(item_path)
2176:         except Exception:
2177:             raise excinfo
2178:
2179:     shutil.rmtree(path, onexc=remove_readonly)
2180:
2181:
```

add_source_fact_summary (docs/build_complete_guide.py, lines 2182-2195)

add_source_fact_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2182-2195 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_table, fact_relationship_summary, str, and len. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2182: def add_source_fact_summary(doc: Document, facts: dict) -> None:
2183:     add_table(doc, [
2184:         ("File", facts["repo_file"]),
2185:         ("Purpose", facts["description"]),
2186:         ("Lines", f"{facts['line_count']:,}"),
2187:         ("Size", f"{facts['size_bytes']:,} bytes"),
2188:         ("Talks to", fact_relationship_summary(facts)),
2189:         ("Functions", str(len(facts["functions"]))),
2190:         ("Variables/constants", str(len(facts["variables"]))),
2191:         ("API mentions", str(len(facts["endpoints"]))),
2192:         ("Signals", str(len(facts["signals"]))),
2193:     ])
2194:
2195:
```

add_source_fact_sections (docs/build_complete_guide.py, lines 2196-2238)

add_source_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2196-2238 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, endpoint_purpose, add_function_explanation_docx, add_code, and numbers. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2196: def add_source_fact_sections(doc: Document, facts: dict, include_excerpts: bool = True) -> None:
 2197:     doc.add_heading("How this file participates in the system", level=2)
 2198:     doc.add_paragraph("This generated section reads the actual file and points out how it communicates
with other pieces of the app. It is intentionally concrete: line numbers, declarations, endpoints, events,
source excerpts, and debugging cues.")
 2199:     if facts["imports"]:
 2200:         doc.add_heading("Imports, requires, and external dependencies", level=2)
 2201:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["imports"][:60]])
 2202:     if facts["routes"]:
 2203:         doc.add_heading("Backend routes implemented here", level=2)
 2204:         add_table(doc, [(f"{item['method']} {item['path']}", f"Line {item['line']}.
{endpoint_purpose(item['path'])}") for item in facts["routes"][:80]])
 2205:     if facts["endpoints"]:
 2206:         doc.add_heading("API endpoints mentioned or called", level=2)
 2207:         add_table(doc, [(item["endpoint"], f"Line {item['line']}. {endpoint_purpose(item['endpoint'])}")
for item in facts["endpoints"][:80]])
 2208:     if facts["signals"]:
 2209:         doc.add_heading("Important implementation signals", level=2)
 2210:         add_table(doc, [(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]])
 2211:     if facts["variables"]:
 2212:         doc.add_heading("Important constants and variables", level=2)
 2213:         add_table(doc, [(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]])
 2214:     if facts["json_keys"]:
 2215:         doc.add_heading("JSON/YAML-style keys detected", level=2)
 2216:         add_table(doc, [(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]])
 2217:     if facts["events"]:
 2218:         doc.add_heading("Event handlers and user interactions", level=2)
 2219:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["events"][:80]])
 2220:     if facts["selectors"]:
 2221:         doc.add_heading("CSS selectors and visual hooks", level=2)
 2222:         add_table(doc, [(f"Line {item['line']}", item["selector"]) for item in facts["selectors"][:120]])
 2223:     if facts["functions"]:
 2224:         doc.add_heading("Function-by-function detail", level=2)
 2225:         for item in facts["functions"]:
 2226:             add_function_explanation_docx(doc, facts["repo_file"], item,
include_excerpt=include_excerpts, heading_level=3)
 2227:     doc.add_heading("Representative opening snippet", level=2)
 2228:     add_code(doc, facts["snippet"] or "(empty file)")
 2229:     doc.add_heading("Beginner debugging checklist for this file", level=2)
 2230:     numbers(doc, [
 2231:         "Name the user action, command, or workflow that reaches this file.",
 2232:         "Find the exact function or route that owns the behavior.",
 2233:         "Check the inputs: DOM state, request body, file path, local storage value, compiler source, or
Git branch.",
 2234:         "Check the outputs: returned JSON, updated DOM, written file, terminal output, generated project
catalog, or published website asset.",
 2235:         "If the issue crosses a boundary, test the boundary directly: browser console for frontend,
endpoint call for backend, command line for Git/compiler, Cloudflare logs for Worker behavior.",
 2236:     ])
 2237:
 2238:
```

setup_code_reference_document (docs/build_complete_guide.py, lines 2239-2252)

setup_code_reference_document named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2239-2252 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_document, add_paragraph, add_run, Pt, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2250: return doc.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2239: def setup_code_reference_document(title: str, subtitle: str) -> Document:
2240:     doc = setup_document()
2241:     title_p = doc.add_paragraph()
2242:     title_p.alignment = WD_ALIGN_PARAGRAPH.LEFT
2243:     run = title_p.add_run(title)
2244:     run.bold = True
2245:     run.font.size = Pt(22)
2246:     run.font.color.rgb = RGBColor(11, 37, 69)
2247:     doc.add_paragraph(subtitle)
2248:     doc.add_paragraph(f"Generated from repository state at {REPO}.")
2249:     doc.add_paragraph("")
2250:     return doc
2251:
2252:
```

compact_generated_docx (docs/build_complete_guide.py, lines 2253-2273)

compact_generated_docx named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2253-2273 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references list, any, qn, get, iter, getparent, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2271: return removed.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2253: def compact_generated_docx(doc: Document, keep_first_page_break: bool = True) -> int:
2254:     """Remove generator-inserted page breaks so pages fill naturally."""
2255:     kept = 0
2256:     removed = 0
2257:     for paragraph in list(doc.paragraphs):
2258:         has_page_break = any(
2259:             element.tag == qn("w:br") and element.get(qn("w:type")) == "page"
2260:             for element in paragraph._element.iter()
2261:         )
2262:         if not has_page_break:
2263:             continue
2264:         if keep_first_page_break and kept == 0:
2265:             kept += 1
2266:             continue
2267:         parent = paragraph._element.getparent()
2268:         if parent is not None:
2269:             parent.remove(paragraph._element)
2270:             removed += 1
2271:     return removed
2272:
2273:
```

write_single_code_reference_docx (docs/build_complete_guide.py, lines 2274-2284)

write_single_code_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2274-2284 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_source_fact_summary, add_source_fact_sections, compact_generated_docx, and save. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2274: def write_single_code_reference_docx(facts: dict, output_path: Path) -> None:
2275:     doc = setup_code_reference_document(
2276:         f"Code Reference: {facts['repo_file']}",
2277:         "A source-level explanation with function details, file communication, variables, endpoints, and
debugging notes.",
2278:     )
2279:     add_source_fact_summary(doc, facts)
2280:     add_source_fact_sections(doc, facts, include_excerpts=True)
2281:     compact_generated_docx(doc, keep_first_page_break=False)
2282:     doc.save(output_path)
```

```
2283:
2284:
```

pdf_styles (docs/build_complete_guide.py, lines 2285-2314)

pdf_styles named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2285-2314 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSampleStyleSheet, add, ParagraphStyle, and HexColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2312: return styles.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2285: def pdf_styles():
2286:     styles = getSampleStyleSheet()
2287:     styles.add(ParagraphStyle(
2288:         name="CodeTiny",
2289:         parent=styles["Code"],
2290:         fontName="Courier",
2291:         fontSize=6.4,
2292:         leading=7.6,
2293:         backColor=colors.HexColor("#F5F8FB"),
2294:         borderColor=colors.HexColor("#D8E6F0"),
2295:         borderWidth=0.25,
2296:         borderPadding=4,
2297:         spaceAfter=7,
2298:     ))
2299:     styles.add(ParagraphStyle(
2300:         name="SmallBody",
2301:         parent=styles["BodyText"],
2302:         fontSize=8.5,
2303:         leading=11.0,
2304:         spaceAfter=5,
2305:     ))
2306:     styles.add(ParagraphStyle(
2307:         name="TinyTable",
2308:         parent=styles["BodyText"],
2309:         fontSize=7.0,
2310:         leading=8.5,
2311:     ))
2312:     return styles
2313:
2314:
```

pdf_paragraph (docs/build_complete_guide.py, lines 2315-2318)

pdf_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2315-2318 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Paragraph, escape, str, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2316: return Paragraph(html.escape(str(text)).replace("\n", "
"), style).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2315: def pdf_paragraph(text: str, style) -> Paragraph:
2316:     return Paragraph(html.escape(str(text)).replace("\n", "<br/>"), style)
2317:
2318:
```

pdf_code (docs/build_complete_guide.py, lines 2319-2327)

pdf_code named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2319-2327 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references in, splitlines, replace, wrap, extend, Preformatted, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2325: return Preformatted("\n".join(wrapped_lines), styles["CodeTiny"]).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2319: def pdf_code(text: str, styles) -> Preformatted:
 2320:     wrapped_lines = []
 2321:     for raw_line in (text or "").splitlines():
 2322:         expanded = raw_line.replace("\t", "    ")
 2323:         chunks = textwrap.wrap(expanded, width=112, replace_whitespace=False, drop_whitespace=False) or
[""]
 2324:         wrapped_lines.extend(chunks)
 2325:     return Preformatted("\n".join(wrapped_lines), styles["CodeTiny"])
 2326:
 2327:
```

pdf_table (docs/build_complete_guide.py, lines 2328-2343)

pdf_table named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2328-2343 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_paragraph, extend, Table, setStyle, TableStyle, and HexColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2341: return table.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2328: def pdf_table(rows: list[tuple[str, str]], styles):
 2329:     data = [[pdf_paragraph("Item", styles["TinyTable"]), pdf_paragraph("Explanation",
styles["TinyTable"])]]
 2330:     data.extend([[pdf_paragraph(label, styles["TinyTable"]), pdf_paragraph(value, styles["TinyTable"])]
for label, value in rows])
 2331:     table = Table(data, colWidths=[1.45 * inch, 5.25 * inch], repeatRows=1)
 2332:     table.setStyle(TableStyle([
 2333:         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#E8EEF5")),
 2334:         ("GRID", (0, 0), (-1, -1), 0.25, colors.HexColor("#B7CFE0")),
 2335:         ("VALIGN", (0, 0), (-1, -1), "TOP"),
 2336:         ("LEFTPADDING", (0, 0), (-1, -1), 5),
 2337:         ("RIGHTPADDING", (0, 0), (-1, -1), 5),
 2338:         ("TOPPADDING", (0, 0), (-1, -1), 4),
 2339:         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
 2340:     ]))
 2341:     return table
 2342:
 2343:
```

add_pdf_source_fact_sections (docs/build_complete_guide.py, lines 2344-2378)

add_pdf_source_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2344-2378 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, Paragraph, pdf_paragraph, pdf_table, endpoint_purpose, add_function_explanation_pdf, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2344: def add_pdf_source_fact_sections(story: list, facts: dict, styles, include_excerpts: bool = True) ->
None:
 2345:     story.append(Paragraph("How this file participates in the system", styles["Heading2"]))
 2346:     story.append(pdf_paragraph("This generated section reads the actual file and points out line numbers,
 declarations, endpoints, events, source excerpts, and debugging cues.", styles["SmallBody"]))
```

```
    if facts["imports"]:
        story.append(Paragraph("Imports, requires, and external dependencies", styles["Heading2"]))
        story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["imports"][:60]], styles))
    if facts["routes"]:
        story.append(Paragraph("Backend routes implemented here", styles["Heading2"]))
        story.append(pdf_table([(f"{item['method']} {item['path']}", f"Line {item['line']}.
{endpoint_purpose(item['path'])}") for item in facts["routes"][:80]], styles))
    if facts["endpoints"]:
        story.append(Paragraph("API endpoints mentioned or called", styles["Heading2"]))
        story.append(pdf_table([(item["endpoint"], f"Line {item['line']}.
{endpoint_purpose(item['endpoint'])}") for item in facts["endpoints"][:80]], styles))
    if facts["signals"]:
        story.append(Paragraph("Important implementation signals", styles["Heading2"]))
        story.append(pdf_table([(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]], styles))
    if facts["variables"]:
        story.append(Paragraph("Important constants and variables", styles["Heading2"]))
        story.append(pdf_table([(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]], styles))
    if facts["json_keys"]:
        story.append(Paragraph("JSON/YAML-style keys detected", styles["Heading2"]))
        story.append(pdf_table([(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]], styles))
    if facts["events"]:
        story.append(Paragraph("Event handlers and user interactions", styles["Heading2"]))
        story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["events"][:80]], styles))
    if facts["selectors"]:
        story.append(Paragraph("CSS selectors and visual hooks", styles["Heading2"]))
        story.append(pdf_table([(f"Line {item['line']}", item["selector"]) for item in
facts["selectors"][:120]], styles))
    if facts["functions"]:
        story.append(Paragraph("Function-by-function detail", styles["Heading2"]))
        for item in facts["functions"]:
            add_function_explanation_pdf(story, facts["repo_file"], item, styles,
include_excerpt=include_excerpts)
    story.append(Paragraph("Representative opening snippet", styles["Heading2"]))
    story.append(pdf_code(facts["snippet"] or "(empty file)", styles))


```

write_single_code_reference_pdf (docs/build_complete_guide.py, lines 2379-2402)

write_single_code_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2379-2402 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Paragraph, pdf_paragraph, pdf_table, fact_relationship_summary, str, len, Spacer, add_pdf_source_fact_sections, SimpleDocTemplate, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2381: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def write_single_code_reference_pdf(facts: dict, output_path: Path) -> None:
    if not REPORTLAB_AVAILABLE:
        return
    styles = pdf_styles()
    story = [
        Paragraph(f"Code Reference: {facts['repo_file']}", styles["Title"]),
        pdf_paragraph("A source-level explanation with function details, file communication, variables,
endpoints, and debugging notes.", styles["SmallBody"]),
        pdf_table([
            ("File", facts["repo_file"]),
            ("Purpose", facts["description"]),
            ("Lines", f"{facts['line_count']:,}"),
            ("Size", f"{facts['size_bytes']:,} bytes"),
            ("Talks to", fact_relationship_summary(facts)),
            ("Functions", str(len(facts["functions"]))),
            ("Variables/constants", str(len(facts["variables"]))),
            ("API mentions", str(len(facts["endpoints"]))),
            ("Signals", str(len(facts["signals"]))),
        ], styles),
        Spacer(1, 0.12 * inch),
    ]
    add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
    SimpleDocTemplate(str(output_path), pagesize=LETTER, rightMargin=0.55 * inch, leftMargin=0.55 * inch,
topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)


```

write_master_code_reference_docx (docs/build_complete_guide.py, lines 2403-2431)

write_master_code_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2403-2431 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, add_heading, numbers, add_table, len, signal, add_page_break, add_source_fact_summary, add_source_fact_sections, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2403: def write_master_code_reference_docx(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
 2404:     doc = setup_code_reference_document(
 2405:         "OMB Portfolio Builder Code Reference",
 2406:         "A generated Word reference that explains the important source files, all discovered functions,
communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
 2407:     )
 2408:     add_diagram(doc, diagrams["file-communication-map"], "Main source file communication map.")
 2409:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Frontend, backend, Cloudflare, and AI
boundary.")
 2410:     add_diagram(doc, diagrams["compile-code-flow"], "Compile Code workspace and simulator flow.")
 2411:     doc.add_heading("How to use this document", level=1)
 2412:     numbers(doc, [
 2413:         "Start with the file you want to understand.",
 2414:         "Read the summary table to understand its role.",
 2415:         "Read implementation signals to see whether the file touches Git, files, compilers, Cloudflare,
authentication, DOM, storage, or network calls.",
 2416:         "Read function details to understand line ranges, side effects, calls, endpoints, and source
excerpts.",
 2417:         "Use the debugging checklist to trace a behavior from click to function to file/API/output.",
 2418:     ])
 2419:     doc.add_heading("Generated file index", level=1)
 2420:     add_table(doc, [(repo_file, f"{facts_by_file[repo_file]['line_count']:,} lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files])
 2421:     doc.add_page_break()
 2422:     for repo_file in code_files:
 2423:         facts = facts_by_file[repo_file]
 2424:         doc.add_heading(f"Source File: {repo_file}", level=1)
 2425:         add_source_fact_summary(doc, facts)
 2426:         add_source_fact_sections(doc, facts, include_excerpts=True)
 2427:         doc.add_page_break()
 2428:     compact_generated_docx(doc, keep_first_page_break=True)
 2429:     doc.save(CODE_REFERENCE_MASTER_DOCX)
 2430:
 2431:
```

write_master_code_reference_pdf (docs/build_complete_guide.py, lines 2432-2470)

write_master_code_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2432-2470 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, len, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2434: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2432: def write_master_code_reference_pdf(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
 2433:     if not REPORTLAB_AVAILABLE:
 2434:         return
 2435:     styles = pdf_styles()
 2436:     story = [
 2437:         Paragraph("OMB Portfolio Builder Code Reference", styles["Title"]),
 2438:         pdf_paragraph("A generated PDF reference that explains the important source files, discovered
functions, communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
styles["SmallBody"]),
 2439:     ]
```

```
    for diagram_name, caption in [
        ("file-communication-map", "Main source file communication map."),
        ("frontend-backend-cloudflare", "Frontend, backend, Cloudflare, and AI boundary."),
        ("compile-code-flow", "Compile Code workspace and simulator flow."),
    ]:
        path = diagrams.get(diagram_name)
        if path and path.exists():
            story.append(PdfImage(str(path), width=6.6 * inch, height=3.72 * inch))
            story.append(pdf_paragraph(caption, styles["SmallBody"]))
    story.append(Paragraph("Generated file index", styles["Heading1"]))
    story.append(pdf_table([(repo_file, f"{facts_by_file[repo_file]['line_count']:,} lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files], styles))
    story.append(PageBreak())
    for repo_file in code_files:
        facts = facts_by_file[repo_file]
        story.append(Paragraph(f"Source File: {repo_file}", styles["Heading1"]))
        story.append(pdf_table([
            ("File", facts["repo_file"]),
            ("Purpose", facts["description"]),
            ("Lines", f"{facts['line_count']:,}"),
            ("Size", f"{facts['size_bytes']:,} bytes"),
            ("Talks to", fact_relationship_summary(facts)),
            ("Functions", str(len(facts["functions"]))),
            ("Variables/constants", str(len(facts["variables"]))),
            ("API mentions", str(len(facts["endpoints"]))),
            ("Signals", str(len(facts["signals"]))),
        ], styles))
        add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
        story.append(PageBreak())
    SimpleDocTemplate(str(CODE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)


```

write_single_file_reference_docx (docs/build_complete_guide.py, lines 2471-2475)

write_single_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2471-2475 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def write_single_file_reference_docx(
    facts: dict,
    output_path: Path,
    title_prefix: str = "File Reference",
    subtitle: str = "A file-specific explanation covering purpose, owner layer, metadata, source details,
implementation signals, source excerpts, and safe-change rules.",
```

add_pdf_file_fact_sections (docs/build_complete_guide.py, lines 2487-2513)

add_pdf_file_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2487-2513 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, Paragraph, pdf_paragraph, add_pdf_source_fact_sections, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def add_pdf_file_fact_sections(story: list, facts: dict, styles) -> None:
    story.append(Paragraph("Why this file exists", styles["Heading2"]))
    for note in facts["notes"]:
        story.append(pdf_paragraph(f"- {note}", styles["SmallBody"]))
    story.append(Paragraph("How it is used", styles["Heading2"]))
    story.append(pdf_paragraph(facts["folder_role"], styles["SmallBody"]))
    if facts["source"]:
```

```
 2494:         add_pdf_source_fact_sections(story, facts["source"], styles, include_excerpts=True)
 2495:     elif facts["snippet"]:
 2496:         story.append(Paragraph("Readable content snippet", styles["Heading2"]))
 2497:         story.append(pdf_code(facts["snippet"], styles))
 2498:     elif facts["image"]:
 2499:         story.append(Paragraph("Asset handling note", styles["Heading2"]))
 2500:         story.append(pdf_paragraph("This file is documented by metadata instead of raw bytes. Binary
image/icon bytes are not useful to print in a reference document.", styles["SmallBody"]))
 2501:     else:
 2502:         story.append(Paragraph("Binary or marker note", styles["Heading2"]))
 2503:         story.append(pdf_paragraph("This file is either binary, empty, or a marker/config file whose
value comes from its presence and path rather than readable content.", styles["SmallBody"]))
 2504:     story.append(Paragraph("Safe change checklist", styles["Heading2"]))
 2505:     for item in [
 2506:         "Confirm which app, website, installer, workflow, or document consumes this file.",
 2507:         "Change it on a feature branch from development.",
 2508:         "Regenerate docs if the file purpose, API surface, or behavior changes.",
 2509:         "Run the smallest relevant smoke test before merging into development.",
 2510:     ]:
 2511:         story.append(pdf_paragraph(f"- {item}", styles["SmallBody"]))
 2512:
 2513:
```

write_single_file_reference_pdf (docs/build_complete_guide.py, lines 2514-2518)

write_single_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2514-2518 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2514: def write_single_file_reference_pdf(
 2515:     facts: dict,
 2516:     output_path: Path,
 2517:     title_prefix: str = "File Reference",
 2518:     subtitle: str = "A file-specific explanation covering purpose, owner layer, metadata, source details,
 implementation signals, source excerpts, and safe-change rules.",
```

write_master_file_reference_docx (docs/build_complete_guide.py, lines 2548-2572)

write_master_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2548-2572 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, Counter, add_heading, add_table, str, len, join, sorted, items, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2548: def write_master_file_reference_docx(file_facts: list[dict], diagrams: dict[str, Path]) -> None:
 2549:     doc = setup_code_reference_document(
 2550:         "OMB Portfolio Builder Important Code Reference",
 2551:         "A generated Word reference for the code and control files that drive the builder, website,
installer, Cloudflare AI worker, workflows, parser, compile workspace, and generated documentation.",
 2552:     )
 2553:     add_diagram(doc, diagrams["file-communication-map"], "Main file communication map for the builder and
 public website.")
 2554:     category_counts = Counter(facts["category"] for facts in file_facts)
 2555:     doc.add_heading("Coverage summary", level=1)
 2556:     add_table(doc, [
 2557:         ("Important files documented", str(len(file_facts))),
 2558:         ("Selection rule", "Only behavior-driving code/control files are included. Favicons, binary
assets, and passive marker files are intentionally skipped here."),
 2559:         ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
 2560:     ])
 2561:     doc.add_heading("File index", level=1)
 2562:     add_table(doc, [(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for facts in
file_facts])
```

```
    doc.add_page_break()
    for facts in file_facts:
        doc.add_heading(f"Important Code Reference: {facts['repo_file']}", level=1)
        add_file_fact_summary(doc, facts)
        add_file_fact_sections(doc, facts)
        doc.add_page_break()
    compact_generated_docx(doc, keep_first_page_break=True)
    doc.save(FILE_REFERENCE_MASTER_DOCX)


```

write_master_file_reference_pdf (docs/build_complete_guide.py, lines 2573-2613)

write_master_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2573-2613 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Counter, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2575: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
def write_master_file_reference_pdf(file_facts: list[dict], diagrams: dict[str, Path]) -> None:
    if not REPORTLAB_AVAILABLE:
        return
    styles = pdf_styles()
    category_counts = Counter(facts["category"] for facts in file_facts)
    story = [
        Paragraph("OMB Portfolio Builder Important Code Reference", styles["Title"]),
        pdf_paragraph("A generated PDF reference for the code and control files that drive the builder,
website, installer, Cloudflare AI worker, workflows, parser, compile workspace, and generated documentation.",
styles["SmallBody"]),
    ]
    file_map = diagrams.get("file-communication-map")
    if file_map and file_map.exists():
        story.append(PdfImage(str(file_map), width=6.6 * inch, height=3.72 * inch))
    story.append(Paragraph("Coverage summary", styles["Heading1"]))
    story.append(pdf_table([
        ("Important files documented", str(len(file_facts))),
        ("Selection rule", "Only behavior-driving code/control files are included. Favicons, binary
assets, and passive marker files are intentionally skipped here."),
        ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
    ], styles))
    story.append(Paragraph("File index", styles["Heading1"]))
    story.append(pdf_table([(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for
facts in file_facts], styles))
    story.append(PageBreak())
    for facts in file_facts:
        story.append(Paragraph(f"Important Code Reference: {facts['repo_file']}", styles["Heading1"]))
        rows = [
            ("File", facts["repo_file"]),
            ("Category", facts["category"]),
            ("Folder role", facts["folder_role"]),
            ("Size", f"{facts['size_bytes']:,} bytes"),
            ("Extension", facts["suffix"]),
            ("MIME type", facts["mime_type"]),
            ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text
file"),
        ]
        if facts["image"]:
            image = facts["image"]
            rows.append(("Image metadata", f"Could not inspect image: {image['error']}" if "error" in
image else f"{image['format']} {image['width']}x{image['height']}, mode {image['mode']}, frames
{image['frames']}"))
        story.append(pdf_table(rows, styles))
        add_pdf_file_fact_sections(story, facts, styles)
        story.append(PageBreak())
    SimpleDocTemplate(str(FILE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)


```

write_master_repository_file_reference_docx (docs/build_complete_guide.py, lines 2614-2645)

write_master_repository_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2614-2645 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, Counter, add_heading, add_table, str, len, join, sorted, items, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2614: def write_master_repository_file_reference_docx(file_facts: list[dict], diagrams: dict[str, Path]) ->
None:
 2615:     doc = setup_code_reference_document(
 2616:         "OMB Portfolio Builder Repository File Reference",
 2617:         "A generated Word reference with one explanation section for each primary tracked repository
file. Source files receive function-level detail; assets and marker files receive metadata, purpose, usage, and
safe-change notes.",
 2618:     )
 2619:     add_diagram(doc, diagrams["file-communication-map"], "Main file communication map for the builder and
 public website.")
 2620:     category_counts = Counter(facts["category"] for facts in file_facts)
 2621:     doc.add_heading("Coverage summary", level=1)
 2622:     add_table(doc, [
 2623:         ("Primary files documented", str(len(file_facts))),
 2624:         ("Selection rule", "All primary tracked repository files are included. Generated reference
documents, generated guide diagrams, and generated documentation folders are excluded so this reference does not
 recursively document its own output."),
 2625:         ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
 2626:     ])
 2627:     doc.add_heading("How to use this reference", level=1)
 2628:     numbers(doc, [
 2629:         "Open the specific file document when you want to understand one file in isolation.",
 2630:         "Use this master reference when you want to scan all files from one document.",
 2631:         "For source files, read the function-by-function detail to understand what the function does,
what it calls, what it returns, important local variables, and why it exists.",
 2632:         "For assets, marker files, and static configuration, read the metadata and safe-change checklist
instead of looking for function explanations that do not exist.",
 2633:     ])
 2634:     doc.add_heading("File index", level=1)
 2635:     add_table(doc, [(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for facts in
file_facts])
 2636:     doc.add_page_break()
 2637:     for facts in file_facts:
 2638:         doc.add_heading(f"File Reference: {facts['repo_file']}", level=1)
 2639:         add_file_fact_summary(doc, facts)
 2640:         add_file_fact_sections(doc, facts)
 2641:         doc.add_page_break()
 2642:     compact_generated_docx(doc, keep_first_page_break=True)
 2643:     doc.save(ALL_FILE_REFERENCE_MASTER_DOCX)
 2644:
 2645:
```

write_master_repository_file_reference_pdf (docs/build_complete_guide.py, lines 2646-2686)

write_master_repository_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2646-2686 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Counter, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2648: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2646: def write_master_repository_file_reference_pdf(file_facts: list[dict], diagrams: dict[str, Path]) ->
None:
 2647:     if not REPORTLAB_AVAILABLE:
 2648:         return
 2649:     styles = pdf_styles()
 2650:     category_counts = Counter(facts["category"] for facts in file_facts)
 2651:     story = [
 2652:         Paragraph("OMB Portfolio Builder Repository File Reference", styles["Title"]),
 2653:         pdf_paragraph("A generated PDF reference with one explanation section for each primary tracked
repository file. Source files receive function-level detail; assets and marker files receive metadata, purpose,
```

```
usage, and safe-change notes.", styles["SmallBody"]),
 2654:     ]
 2655:     file_map = diagrams.get("file-communication-map")
 2656:     if file_map and file_map.exists():
 2657:         story.append(PdfImage(str(file_map), width=6.6 * inch, height=3.72 * inch))
 2658:     story.append(Paragraph("Coverage summary", styles["Heading1"]))
 2659:     story.append(pdf_table([
 2660:         ("Primary files documented", str(len(file_facts))),
 2661:         ("Selection rule", "All primary tracked repository files are included. Generated reference
outputs are excluded to avoid recursive documentation."),
 2662:         ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
 2663:     ], styles))
 2664:     story.append(Paragraph("File index", styles["Heading1"]))
 2665:     story.append(pdf_table([(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for
facts in file_facts], styles))
 2666:     story.append(PageBreak())
 2667:     for facts in file_facts:
 2668:         story.append(Paragraph(f"File Reference: {facts['repo_file']}", styles["Heading1"]))
 2669:         rows = [
 2670:             ("File", facts["repo_file"]),
 2671:             ("Category", facts["category"]),
 2672:             ("Folder role", facts["folder_role"]),
 2673:             ("Size", f"{facts['size_bytes']:,} bytes"),
 2674:             ("Extension", facts["suffix"]),
 2675:             ("MIME type", facts["mime_type"]),
 2676:             ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text
file"),
 2677:         ]
 2678:         if facts["image"]:
 2679:             image = facts["image"]
 2680:             rows.append(("Image metadata", f"Could not inspect image: {image['error']}" if "error" in
image else f"{image['format']} {image['width']}x{image['height']}, mode {image['mode']}, frames
{image['frames']}"))
 2681:         story.append(pdf_table(rows, styles))
 2682:         add_pdf_file_fact_sections(story, facts, styles)
 2683:         story.append(PageBreak())
 2684:     SimpleDocTemplate(str(ALL_FILE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
 2685:
 2686:
```

write_repository_file_reference_docs (docs/build_complete_guide.py, lines 2687-2709)

write_repository_file_reference_docs persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2687-2709 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove_directory, mkdir, extract_file_facts, primary_tracked_files, code_reference_name, write_single_file_reference_docx, append, write_single_file_reference_pdf, exists, write_master_repository_file_reference_docx, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2707: return written.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2687: def write_repository_file_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
 2688:     remove_directory(ALL_FILE_REFERENCE_DOCX_DIR)
 2689:     remove_directory(ALL_FILE_REFERENCE_PDF_DIR)
 2690:     ALL_FILE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
 2691:     ALL_FILE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
 2692:     file_facts = [extract_file_facts(file) for file in primary_tracked_files(files)]
 2693:     written: list[Path] = []
 2694:     for facts in file_facts:
 2695:         docx_path = ALL_FILE_REFERENCE_DOCX_DIR / code_reference_name(facts["repo_file"], ".docx")
 2696:         write_single_file_reference_docx(facts, docx_path)
 2697:         written.append(docx_path)
 2698:         pdf_path = ALL_FILE_REFERENCE_PDF_DIR / code_reference_name(facts["repo_file"], ".pdf")
 2699:         write_single_file_reference_pdf(facts, pdf_path)
 2700:         if pdf_path.exists():
 2701:             written.append(pdf_path)
 2702:     write_master_repository_file_reference_docx(file_facts, diagrams)
 2703:     written.append(ALL_FILE_REFERENCE_MASTER_DOCX)
 2704:     write_master_repository_file_reference_pdf(file_facts, diagrams)
 2705:     if ALL_FILE_REFERENCE_MASTER_PDF.exists():
 2706:         written.append(ALL_FILE_REFERENCE_MASTER_PDF)
 2707:     return written
 2708:
 2709:
```

write_file_reference_docs (docs/build_complete_guide.py, lines 2710-2742)

This function regenerates the focused important-code reference DOCX/PDF artifacts for the selected behavior-driving files.

It gives the user a smaller curated reference that is easier to read than the exhaustive per-file set.

It calls or references remove_directory, mkdir, extract_file_facts, important_code_files, code_reference_name, write_single_file_reference_docx, append, write_single_file_reference_pdf, exists, write_master_file_reference_docx, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2740: return written.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2710: def write_file_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
2711:     remove_directory(FILE_REFERENCE_DOCX_DIR)
2712:     remove_directory(FILE_REFERENCE_PDF_DIR)
2713:     FILE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
2714:     FILE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
2715:     file_facts = [extract_file_facts(file) for file in important_code_files(files)]
2716:     written: list[Path] = []
2717:     for facts in file_facts:
2718:         docx_path = FILE_REFERENCE_DOCX_DIR / code_reference_name(facts["repo_file"], ".docx")
2719:         write_single_file_reference_docx(
2720:             facts,
2721:             docx_path,
2722:             title_prefix="Important Code Reference",
2723:             subtitle="A focused code/control-file explanation covering purpose, owner layer, APIs,
variables, implementation signals, source excerpts, and debugging rules.",
2724:         )
2725:         written.append(docx_path)
2726:         pdf_path = FILE_REFERENCE_PDF_DIR / code_reference_name(facts["repo_file"], ".pdf")
2727:         write_single_file_reference_pdf(
2728:             facts,
2729:             pdf_path,
2730:             title_prefix="Important Code Reference",
2731:             subtitle="A focused code/control-file explanation covering purpose, owner layer, APIs,
variables, implementation signals, source excerpts, and debugging rules.",
2732:         )
2733:         if pdf_path.exists():
2734:             written.append(pdf_path)
2735:     write_master_file_reference_docx(file_facts, diagrams)
2736:     written.append(FILE_REFERENCE_MASTER_DOCX)
2737:     write_master_file_reference_pdf(file_facts, diagrams)
2738:     if FILE_REFERENCE_MASTER_PDF.exists():
2739:         written.append(FILE_REFERENCE_MASTER_PDF)
2740:     return written
2741:
2742:
```

write_code_reference_docs (docs/build_complete_guide.py, lines 2743-2768)

This function regenerates the exhaustive per-file code-reference DOCX/PDF artifacts from the current repository state.

It keeps the file-specific docs synchronized with the actual code whenever the repository changes.

It calls or references remove_directory, mkdir, is_text_code_file, extract_source_facts, code_reference_name, write_single_code_reference_docx, append, write_single_code_reference_pdf, exists, write_master_code_reference_docx, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2766: return written.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2743: def write_code_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
2744:     remove_directory(CODE_REFERENCE_DIR)
2745:     remove_directory(CODE_REFERENCE_DOCX_DIR)
2746:     remove_directory(CODE_REFERENCE_PDF_DIR)
2747:     CODE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
2748:     CODE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
2749:     code_files = [file for file in files if is_text_code_file(file)]
2750:     facts_by_file = {repo_file: extract_source_facts(repo_file) for repo_file in code_files}
2751:     written: list[Path] = []
2752:     for repo_file in code_files:
2753:         facts = facts_by_file[repo_file]
2754:         docx_path = CODE_REFERENCE_DOCX_DIR / code_reference_name(repo_file, ".docx")
2755:         write_single_code_reference_docx(facts, docx_path)
2756:         written.append(docx_path)
2757:         pdf_path = CODE_REFERENCE_PDF_DIR / code_reference_name(repo_file, ".pdf")
2758:         write_single_code_reference_pdf(facts, pdf_path)
2759:         if pdf_path.exists():
```

```
 2760:             written.append(pdf_path)
 2761:     write_master_code_reference_docx(code_files, facts_by_file, diagrams)
 2762:     written.append(CODE_REFERENCE_MASTER_DOCX)
 2763:     write_master_code_reference_pdf(code_files, facts_by_file, diagrams)
 2764:     if CODE_REFERENCE_MASTER_PDF.exists():
 2765:         written.append(CODE_REFERENCE_MASTER_PDF)
 2766:     return written
 2767:
 2768:
```

add_generated_code_reference (docs/build_complete_guide.py, lines 2769-2789)

add_generated_code_reference named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2769-2789 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, write_code_reference_docs, add_paragraph, add_table, str, relative_to, exists, len, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2769: def add_generated_code_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
 2770:     doc.add_heading("Generated Word And PDF Code References", level=1)
 2771:     written = write_code_reference_docs(files, diagrams)
 2772:     doc.add_paragraph("The repository includes generated Word and PDF code-reference documents for
detailed per-file study. The complete guide intentionally does not duplicate every function from every file. Use
 the file-specific references when you want exhaustive function coverage for a particular source file.")
 2773:     add_table(doc, [
 2774:         ("Word folder", str(CODE_REFERENCE_DOCX_DIR.relative_to(REPO))),
 2775:         ("PDF folder", str(CODE_REFERENCE_PDF_DIR.relative_to(REPO))),
 2776:         ("Master Word reference", str(CODE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
 2777:         ("Master PDF reference", str(CODE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
CODE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
 2778:         ("Artifacts generated", str(len(written))),
 2779:         ("Regeneration command", "python docs/build_complete_guide.py"),
 2780:     ])
 2781:     doc.add_heading("What belongs here versus file-specific docs", level=2)
 2782:     bullets(doc, [
 2783:         "The complete guide explains architecture, workflows, boundaries, tools, and how the whole system
 fits together.",
 2784:         "The code-reference DOCX/PDF files explain every discovered function inside each specific file.",
 2785:         "When editing one file, open that file's generated reference rather than scrolling through the
complete guide.",
 2786:     ])
 2787:     doc.add_page_break()
 2788:
 2789:
```

add_generated_repository_file_reference (docs/build_complete_guide.py, lines 2790-2813)

add_generated_repository_file_reference named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2790-2813 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, write_repository_file_reference_docs, primary_tracked_files, add_paragraph, add_table, str, relative_to, exists, len, bullets, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2790: def add_generated_repository_file_reference(doc: Document, files: list[str], diagrams: dict[str, Path])
-> None:
 2791:     doc.add_heading("Generated Word And PDF Repository File References", level=1)
 2792:     written = write_repository_file_reference_docs(files, diagrams)
 2793:     documented_files = primary_tracked_files(files)
 2794:     doc.add_paragraph("The repository now has a file-specific Word document and PDF for each primary
tracked file. Source files receive detailed function explanations. Static assets, favicons, marker files,
```

```
security files, workflows, JSON files, and text documents receive purpose, metadata, usage, and safe-change
notes.")
 2795:     add_table(doc, [
 2796:         ("Word folder", str(ALL_FILE_REFERENCE_DOCX_DIR.relative_to(REPO))),
 2797:         ("PDF folder", str(ALL_FILE_REFERENCE_PDF_DIR.relative_to(REPO))),
 2798:         ("Master Word reference", str(ALL_FILE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
 2799:         ("Master PDF reference", str(ALL_FILE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
ALL_FILE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
 2800:         ("Primary files documented", str(len(documented_files))),
 2801:         ("Artifacts generated", str(len(written))),
 2802:         ("Excluded", "Generated documentation outputs and generated guide diagrams are skipped to avoid
recursively documenting generated files."),
 2803:         ("Regeneration command", "python docs/build_complete_guide.py"),
 2804:     ])
 2805:     doc.add_heading("How this supports slow curation", level=2)
 2806:     bullets(doc, [
 2807:         "Each file can be improved independently because it now has its own document.",
 2808:         "The complete guide can stay focused on system architecture instead of becoming an unreadable
function dump.",
 2809:         "When a file changes, regenerate the references and review only that file's document first.",
 2810:     ])
 2811:     doc.add_page_break()
 2812:
 2813:
```

add_generated_file_reference (docs/build_complete_guide.py, lines 2814-2837)

add_generated_file_reference named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2814-2837 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, write_file_reference_docs, important_code_files, add_paragraph, add_table, str, relative_to, exists, len, bullets, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2814: def add_generated_file_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
 2815:     doc.add_heading("Generated Word And PDF Important Code References", level=1)
 2816:     written = write_file_reference_docs(files, diagrams)
 2817:     selected_files = important_code_files(files)
 2818:     doc.add_paragraph("The repository now includes generated file-specific Word and PDF documents for the
 important code and control files that explain how the system actually works. This focused set avoids passive
binary assets and marker files so the documentation stays useful for learning the builder internals.")
 2819:     add_table(doc, [
 2820:         ("Word folder", str(FILE_REFERENCE_DOCX_DIR.relative_to(REPO))),
 2821:         ("PDF folder", str(FILE_REFERENCE_PDF_DIR.relative_to(REPO))),
 2822:         ("Master Word reference", str(FILE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
 2823:         ("Master PDF reference", str(FILE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
FILE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
 2824:         ("Important files documented", str(len(selected_files))),
 2825:         ("Artifacts generated", str(len(written))),
 2826:         ("Skipped here", "Favicons, app icons, binary assets, passive marker files, and generated
reference documents."),
 2827:         ("Regeneration command", "python docs/build_complete_guide.py"),
 2828:     ])
 2829:     doc.add_heading("What this adds beyond the code reference", level=2)
 2830:     bullets(doc, [
 2831:         "The selected files are the places a developer actually starts when debugging app behavior.",
 2832:         "Each selected file gets source excerpts, implementation signals, API/DOM/storage clues, and
safe-change notes.",
 2833:         "The master reference acts like a guided map of the app internals rather than a dump of every
asset.",
 2834:     ])
 2835:     doc.add_page_break()
 2836:
 2837:
```

add_complete_function_inventory (docs/build_complete_guide.py, lines 2838-2856)

add_complete_function_inventory named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2838-2856 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references important_code_files, add_heading, add_paragraph, add_table, relative_to, get, folder_role, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2838: def add_complete_function_inventory(doc: Document, files: list[str]) -> None:
2839:     selected_files = important_code_files(files)
2840:     doc.add_heading("Where The File-Level Details Live", level=1)
2841:     doc.add_paragraph("The complete guide explains the whole system: architecture, workflows, layers,
tools, publishing, installer behavior, AI backend, security boundaries, and how files communicate. It
intentionally does not explain every function body. Each important file has its own generated Word/PDF reference
 where that file's functions, calls, returns, variables, source excerpts, and purpose are explained in detail.")
2842:     add_table(doc, [
2843:         ("Complete guide job", "Explain the whole builder and website system without becoming a raw
source-code dump."),
2844:         ("All-file docs job", "Give every primary tracked file its own Word/PDF explanation, including
metadata for assets and detailed content for readable files."),
2845:         ("Source-code docs job", "Explain each source file's discovered functions, variables, calls,
returns, endpoints, and excerpts."),
2846:         ("Important-file docs job", "Focus on the behavior-driving files a developer should read
first."),
2847:         ("All-file docs", f"{ALL_FILE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{ALL_FILE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2848:         ("Master all-file reference", f"{ALL_FILE_REFERENCE_MASTER_DOCX.relative_to(REPO)} and
{ALL_FILE_REFERENCE_MASTER_PDF.relative_to(REPO)}"),
2849:         ("Important file docs", f"{FILE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{FILE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2850:         ("Exhaustive code docs", f"{CODE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{CODE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2851:     ])
2852:     doc.add_heading("Recommended source-reading order", level=2)
2853:     add_table(doc, [(repo_file, FILE_DESCRIPTIONS.get(repo_file, folder_role(repo_file))) for repo_file
in selected_files])
2854:     doc.add_page_break()
2855:
2856:
```

add_deep_detail_topics (docs/build_complete_guide.py, lines 2857-2870)

add_deep_detail_topics named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2857-2870 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2857: def add_deep_detail_topics(doc: Document) -> None:
2858:     for title, body in DEEP_DETAIL_TOPICS:
2859:         doc.add_heading(f"Deep Detail: {title}", level=1)
2860:         doc.add_paragraph(body)
2861:         doc.add_heading("How to use this detail", level=2)
2862:         bullets(doc, [
2863:             "Start with the user-visible behavior.",
2864:             "Find the file that owns that behavior.",
2865:             "Trace the data handoff to the next file, endpoint, cache, or generated artifact.",
2866:             "Change the smallest responsible piece and test the flow end to end.",
2867:         ])
2868:         doc.add_page_break()
2869:
2870:
```

add_programming_syntax (docs/build_complete_guide.py, lines 2871-2880)

add_programming_syntax named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2871-2880 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2871: def add_programming_syntax(doc: Document) -> None:
2872:     for title, snippet, explanation in PROGRAMMING_SYNTAX_LESSONS:
2873:         doc.add_heading(f"Programming Syntax: {title}", level=1)
2874:         add_code(doc, snippet)
2875:         doc.add_paragraph(explanation)
2876:         doc.add_heading("How this appears in the project", level=2)
2877:         doc.add_paragraph("You will see this pattern in main.cjs, server.mjs, template-preview.js,
script.js, Cloudflare Worker code, HTML files, CSS files, JSON catalogs, and GitHub workflow YAML.")
2878:         doc.add_page_break()
2879:
2880:
```

add_shell_syntax (docs/build_complete_guide.py, lines 2881-2890)

add_shell_syntax named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2881-2890 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2881: def add_shell_syntax(doc: Document) -> None:
2882:     for title, snippet, explanation in SHELL_SYNTAX_LESSONS:
2883:         doc.add_heading(f"Shell Or Script Syntax: {title}", level=1)
2884:         add_code(doc, snippet)
2885:         doc.add_paragraph(explanation)
2886:         doc.add_heading("Why this matters", level=2)
2887:         doc.add_paragraph("Shell scripts glue tools together. They are not the app UI, but they install
tools, build releases, deploy workers, scan directories, and enforce rules.")
2888:         doc.add_page_break()
2889:
2890:
```

add_caching_methods (docs/build_complete_guide.py, lines 2891-2915)

add_caching_methods named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2891-2915 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2891: def add_caching_methods(doc: Document, diagrams: dict[str, Path]) -> None:
2892:     doc.add_heading("Caching Methods Used In The Builder And Website", level=1)
2893:     add_diagram(doc, diagrams["compile-code-flow"], "Compile artifact caching happens inside the Compile
Code workspace.")
2894:     doc.add_paragraph("Caching means remembering a result so the app does not repeat slow or annoying
work. This project uses several different caches, each with a different safety boundary.")
2895:     add_table(doc, [(name, f"{where} Purpose: {purpose}") for name, where, purpose in CACHING_METHODS])
2896:     doc.add_heading("Important safety rule", level=2)
2897:     doc.add_paragraph("A cache must be scoped. Publishing authorization is scoped to the same repository,
branch, user/machine scope, and expiration time. Compiler caches are scoped to source code, language, compiler
path, runtime path, and file name. UI caches like update snooze are local convenience data and do not grant
```

```
publishing access.")
 2898:     doc.add_page_break()
 2899:     for name, where, purpose in CACHING_METHODS:
 2900:         doc.add_heading(f"Cache Detail: {name}", level=1)
 2901:         if "Compiler" in name or "Compile" in name or "Terminal" in name or "source" in name:
 2902:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {name}.")
 2903:         elif "Publishing" in name or "Extended trust" in name:
 2904:             add_diagram(doc, diagrams["release-pipeline"], f"Publishing authorization cache supports this
 release/publish workflow.")
 2905:         elif "HTTP" in name:
 2906:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], f"HTTP no-store protects request
freshness across frontend/backend paths.")
 2907:         add_table(doc, [
 2908:             ("Where implemented", where),
 2909:             ("Why it exists", purpose),
 2910:             ("What can go stale", "Cached data can become outdated when files, tools, versions,
repositories, or user choices change."),
 2911:             ("How to refresh", "Use the explicit refresh/rebuild/authenticate action for that feature,
clear the relevant local data, or restart the builder when appropriate."),
 2912:         ])
 2913:         doc.add_page_break()
 2914:
 2915:
```

add_installer_details (docs/build_complete_guide.py, lines 2916-2951)

add_installer_details named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2916-2951 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, numbers, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2916: def add_installer_details(doc: Document, diagrams: dict[str, Path]) -> None:
 2917:     doc.add_heading("How The Installer Works", level=1)
 2918:     add_diagram(doc, diagrams["release-pipeline"], "Installer and update behavior sits at the end of the
release pipeline.")
 2919:     doc.add_paragraph("The installer is produced by electron-builder and customized by NSIS script code
in build/installer.nsh. electron-builder gathers app files and creates the installer shell. NSIS controls
Windows setup pages, duplicate install checks, shortcut behavior, Git installation checks, update behavior, and
uninstall pages.")
 2920:     add_table(doc, [
 2921:         ("electron-builder", "Reads package.json build settings and packages the Electron app."),
 2922:         ("NSIS", "Runs installer wizard logic and custom script functions."),
 2923:         ("installer.nsh", "Adds OMB-specific setup pages, duplicate detection, update handling,
shortcuts, and Git checks."),
 2924:         ("AppData install", "Default per-user install location avoids Program Files admin prompts."),
 2925:         ("Update install", "Existing installation is updated in place instead of creating a duplicate
copy."),
 2926:     ])
 2927:     doc.add_heading("Installer flow", level=2)
 2928:     numbers(doc, [
 2929:         "customInit runs first.",
 2930:         "The installer checks for an existing registered install.",
 2931:         "If a current or newer install exists, setup stops.",
 2932:         "If an older install exists, setup treats the run as an update and uses the existing location.",
 2933:         "If no install exists, setup scans for duplicate builder copies.",
 2934:         "The custom tools page asks about desktop shortcut and publishing tools.",
 2935:         "Files are installed into the chosen per-user location.",
 2936:         "customInstall creates/removes shortcuts and optionally installs Git for Windows.",
 2937:         "The uninstaller has its own page explaining what will and will not be removed.",
 2938:     ])
 2939:     doc.add_page_break()
 2940:     for name, description in INSTALLER_FUNCTIONS:
 2941:         doc.add_heading(f"Installer Function: {name}", level=1)
 2942:         doc.add_paragraph(description)
 2943:         add_table(doc, [
 2944:             ("File", "build/installer.nsh"),
 2945:             ("Syntax family", "NSIS installer scripting"),
 2946:             ("Function job", description),
 2947:             ("Beginner note", "Installer functions run during setup/uninstall, not while the normal
builder UI is being used."),
 2948:         ])
 2949:         doc.add_page_break()
 2950:
 2951:
```

add_function_maps (docs/build_complete_guide.py, lines 2952-2979)

add_function_maps named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2952-2979 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2952: def add_function_maps(doc: Document, diagrams: dict[str, Path]) -> None:
 2953:     for file_name, group_purpose, functions in FUNCTION_GROUPS:
 2954:         doc.add_heading(f"Function Map: {file_name}", level=1)
 2955:         if file_name == "main.cjs":
 2956:             add_diagram(doc, diagrams["electron-runtime"], "Electron startup functions sit inside this
runtime flow.")
 2957:         elif file_name == "server.mjs":
 2958:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], "server.mjs is the local backend
layer in this map.")
 2959:         elif file_name == "template-preview.js":
 2960:             add_diagram(doc, diagrams["builder-to-site-files"], "template-preview.js is the builder
frontend that sends edits toward parser output.")
 2961:         elif file_name == "script.js":
 2962:             add_diagram(doc, diagrams["cloudflare-ai-flow"], "script.js owns the public website behavior,
 including the AI chat request path.")
 2963:         doc.add_paragraph(group_purpose)
 2964:         add_table(doc, [(name, description) for name, description in functions])
 2965:         doc.add_heading("How to use this map", level=2)
 2966:         doc.add_paragraph("When debugging, find the user action first, then find the function group that
owns it. For example, a compile terminal issue starts in template-preview.js, then calls server.mjs, then
returns output back to template-preview.js.")
 2967:         doc.add_page_break()
 2968:         for name, description in functions:
 2969:             doc.add_heading(f"Function Detail: {file_name} -> {name}", level=1)
 2970:             doc.add_paragraph(description)
 2971:             add_table(doc, [
 2972:                 ("File", file_name),
 2973:                 ("Function", name),
 2974:                 ("Responsibility", description),
 2975:                 ("Debug question", "What input does this function receive, what output or state change
does it produce, and what function calls it next?"),
 2976:             ])
 2977:             doc.add_page_break()
 2978:
 2979:
```

add_architecture (docs/build_complete_guide.py, lines 2980-2999)

add_architecture named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2980-2999 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, get, add_diagram, add_code, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2980: def add_architecture(doc: Document, diagrams: dict[str, Path]) -> None:
 2981:     for title, body, diagram in ARCHITECTURE_PAGES:
 2982:         doc.add_heading(title, level=1)
 2983:         doc.add_paragraph(body)
 2984:         diagram_name = ARCHITECTURE_DIAGRAM_BY_TITLE.get(title)
 2985:         if diagram_name:
 2986:             add_diagram(doc, diagrams[diagram_name], f"Generated block diagram: {title}.")
 2987:         doc.add_heading("Text version of the block diagram", level=2)
 2988:         add_code(doc, diagram)
 2989:         doc.add_heading("Beginner explanation", level=2)
 2990:         doc.add_paragraph("Read each arrow as 'this thing talks to the next thing.' The important rule is
 that editing happens locally in the builder, while viewing happens publicly on the website after generated
static files are published.")
```

```
 2991:         doc.add_heading("What can break", level=2)
 2992:         bullets(doc, [
 2993:             "If local builder state is not saved, the parser may not see the latest edits.",
 2994:             "If Git authentication or branch state is wrong, Apply to site cannot safely push.",
 2995:             "If public hosting cache is stale, a phone may show an older copy until refresh or cache
expiry.",
 2996:         ])
 2997:         doc.add_page_break()
 2998:
 2999:
```

add_file_communication (docs/build_complete_guide.py, lines 3000-3022)

add_file_communication named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3000-3022 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3000: def add_file_communication(doc: Document, diagrams: dict[str, Path]) -> None:
 3001:     doc.add_heading("How The Important Files Communicate", level=1)
 3002:     add_diagram(doc, diagrams["file-communication-map"], "Generated map of the main builder files and
public website files.")
 3003:     doc.add_paragraph("This is the file-level mental model. The desktop app starts in main.cjs, the
backend lives in server.mjs, the builder screen is template-preview.html with template-preview.css and template-
preview.js, and the public website is index.html with styles.css, script.js, projects.json, and assets.")
 3004:     add_table(doc, [
 3005:         ("main.cjs -> server.mjs", "Electron starts the backend so the builder UI can call local APIs."),
 3006:         ("main.cjs -> template-preview.html", "Electron opens the builder window and loads the builder
HTML file."),
 3007:         ("template-preview.html -> template-preview.js", "The builder HTML loads the JavaScript that
handles editing, saving, previewing, compile code, and publishing UI."),
 3008:         ("template-preview.js -> server.mjs", "The builder uses fetch requests to ask the backend to save
 files, parse projects, run Git, run compilers, or publish."),
 3009:         ("server.mjs -> projects.json", "The parser writes public project data used by the website."),
 3010:         ("index.html -> script.js/styles.css", "The public page loads its behavior and visual design."),
 3011:         ("script.js -> projects.json", "The public website reads the generated catalog to render project
windows."),
 3012:     ])
 3013:     doc.add_heading("Why this communication shape is useful", level=2)
 3014:     bullets(doc, [
 3015:         "The public website can stay static and fast.",
 3016:         "The builder can safely use local-only features without exposing them online.",
 3017:         "The parser creates a clean boundary between editable state and visitor-facing output.",
 3018:         "The same project data can be previewed locally before it is published.",
 3019:     ])
 3020:     doc.add_page_break()
 3021:
 3022:
```

add_tool_build_map (docs/build_complete_guide.py, lines 3023-3032)

add_tool_build_map named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3023-3032 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3023: def add_tool_build_map(doc: Document, diagrams: dict[str, Path]) -> None:
 3024:     doc.add_heading("What Tool Builds What", level=1)
 3025:     add_diagram(doc, diagrams["tool-build-map"], "Generated map showing which tool owns each part of the
build and release process.")
 3026:     doc.add_paragraph("A common source of confusion is thinking one tool builds everything. In this
```

```
project, each tool has a narrow job. pnpm installs dependencies, Electron runs the app, electron-builder
packages the app, NSIS makes the installer, Git moves files, GitHub Actions automates release builds, Wrangler
deploys Cloudflare Worker code, and compilers run project source code.")
 3027:     add_table(doc, [(tool, f"{builds} Used for: {where}") for tool, builds, where in TOOL_BUILD_ROWS])
 3028:     doc.add_heading("Beginner shortcut", level=2)
 3029:     doc.add_paragraph("When something fails, ask which tool owns that step. Installer failure points
toward NSIS or electron-builder. AI backend failure points toward Cloudflare Worker or Wrangler. Website publish
 failure points toward Git, GitHub, branch state, or authentication.")
 3030:     doc.add_page_break()
 3031:
 3032:
```

add_software_decisions (docs/build_complete_guide.py, lines 3033-3051)

add_software_decisions named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3033-3051 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3033: def add_software_decisions(doc: Document) -> None:
 3034:     for title, body in SOFTWARE_DECISIONS:
 3035:         doc.add_heading(f"Software Engineering Decision: {title}", level=1)
 3036:         doc.add_paragraph(body)
 3037:         add_table(doc, [
 3038:             ("Decision", title),
 3039:             ("Reason", body),
 3040:             ("What it protects", "Predictable publishing, safer public/private separation, easier
updates, clearer debugging, or better user experience."),
 3041:             ("Tradeoff", "The design may add more files or moving parts, but each moving part has a
defined job."),
 3042:         ])
 3043:         doc.add_heading("How to evaluate this later", level=2)
 3044:         bullets(doc, [
 3045:             "If the feature becomes hard to maintain, check whether responsibilities are mixed across too
 many files.",
 3046:             "If users are confused, improve the builder UI or guide rather than exposing implementation
details.",
 3047:             "If publishing becomes risky, tighten the parser and authentication boundary first.",
 3048:         ])
 3049:         doc.add_page_break()
 3050:
 3051:
```

add_system_layers (docs/build_complete_guide.py, lines 3052-3067)

add_system_layers named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3052-3067 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_page_break, add_table, and add_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3052: def add_system_layers(doc: Document, diagrams: dict[str, Path]) -> None:
 3053:     doc.add_heading("AI, Frontend, Backend, Cloudflare, And Secrets", level=1)
 3054:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Generated map of the frontend, backend,
Cloudflare Worker, AI provider, and secret boundary.")
 3055:     doc.add_paragraph("This section separates the major layers so you know where to look when something
breaks. The frontend draws and captures input. The backend performs protected work. Cloudflare hosts the public
AI endpoint. Secrets stay server-side.")
 3056:     doc.add_page_break()
 3057:     for title, body, rows, snippet in SYSTEM_LAYER_SECTIONS:
 3058:         doc.add_heading(title, level=1)
 3059:         doc.add_paragraph(body)
```

```
3060:         add_table(doc, rows)
3061:         doc.add_heading("Representative code or command", level=2)
3062:         add_code(doc, snippet)
3063:         doc.add_heading("Debug question", level=2)
3064:         doc.add_paragraph("Ask which layer owns the problem: did the UI send the request, did the backend
receive it, did the service have the needed secret or tool, and did the response come back in the format the UI
expects?")
3065:         doc.add_page_break()
3066:
3067:
```

add_workflows (docs/build_complete_guide.py, lines 3068-3101)

add_workflows named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3068-3101 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, numbers, add_paragraph, add_page_break, startswith, read_text, get, add_code, join, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3068: def add_workflows(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
3069:     workflows = [
3070:         ("Content Creation Workflow", ["Open the builder.", "Edit profile, front-page text, portfolio
areas, or projects.", "Save project or Save all sections.", "Open preview and inspect the exact public layout.",
 "Save draft.", "Apply to site after the target is authenticated."]),
3071:         ("Project Creation Workflow", ["Click Add project.", "Choose category.", "Choose appearance.",
"Enter title.", "Edit overview and sections.", "Add files, images, links, code blocks, and compile-code
evidence.", "Save project so the parser can build the project preview."]),
3072:         ("Publishing Workflow", ["Enter repository target.", "Authenticate GitHub write access.", "Load
from target if the target already has content.", "Save draft.", "Apply to site.", "Git commits and pushes
generated files.", "GitHub Pages or Cloudflare serves the updated website."]),
3073:         ("Update Workflow", ["The installed app checks GitHub Releases.", "If a newer version exists, the
 app shows an update dialog.", "The updater downloads the installer.", "The app closes itself.", "The installer
updates the existing AppData install.", "The app relaunches after the installer finishes."]),
3074:         ("Compile Code Workflow", ["Create or import a source file.", "Pick the language.", "For
Verilog/SystemVerilog, mark design files as Design and create or import a Testbench file.", "Keep $dumpfile and
$dumpvars in the HDL testbench so the signal scope can draw waveforms.", "Save source.", "Run beautifier if
desired.", "Compile or run.", "Read terminal output, messages log, and HDL scope when available.", "Append the
source code to a project section when ready."]),
3075:     ]
3076:     for title, steps in workflows:
3077:         doc.add_heading(title, level=1)
3078:         if title in {"Content Creation Workflow", "Project Creation Workflow", "Publishing Workflow"}:
3079:             add_diagram(doc, diagrams["builder-to-site-files"], f"Generated flow diagram supporting
{title}.")
3080:         if title == "Update Workflow":
3081:             add_diagram(doc, diagrams["release-pipeline"], "Generated release/update pipeline diagram.")
3082:         if title == "Compile Code Workflow":
3083:             add_diagram(doc, diagrams["compile-code-flow"], "Generated Compile Code workflow diagram.")
3084:         numbers(doc, steps)
3085:         doc.add_heading("Plain-English mental model", level=2)
3086:         doc.add_paragraph("A workflow is just a repeatable recipe. The builder turns complicated actions
into visible buttons so you do not need to remember shell commands every time.")
3087:         doc.add_page_break()
3088:
3089:     for workflow in [f for f in files if f.startswith(".github/workflows/")]:
3090:         text = (REPO / workflow).read_text(encoding="utf-8", errors="replace")
3091:         doc.add_heading(f"GitHub Workflow File: {workflow}", level=1)
3092:         doc.add_paragraph(FILE_DESCRIPTIONS.get(workflow, "GitHub Actions workflow file."))
3093:         doc.add_heading("What a GitHub workflow is", level=2)
3094:         doc.add_paragraph("A workflow is an automated script GitHub runs in the cloud. Instead of you
manually building installers or enforcing branch rules, GitHub reads this YAML file and executes each job on a
temporary machine.")
3095:         doc.add_heading("Important YAML excerpt", level=2)
3096:         add_code(doc, "\n".join(text.splitlines()[:80]))
3097:         doc.add_heading("Tradeoff", level=2)
3098:         doc.add_paragraph("Automated workflows reduce mistakes, but a bad workflow can block good merges.
That is why the main branch gate should be strict but understandable.")
3099:         doc.add_page_break()
3100:
3101:
```

file_type (docs/build_complete_guide.py, lines 3102-3109)

file_type named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3102-3109 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references endswith, lower, and lstrip. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3104: return "folder marker"; line 3106: return path.suffix.lower().lstrip("."); line 3107: return "no extension".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3102: def file_type(path: Path, repo_path: str) -> str:
 3103:     if repo_path.endswith(".gitkeep"):
 3104:         return "folder marker"
 3105:     if path.suffix:
 3106:         return path.suffix.lower().lstrip(".")
 3107:     return "no extension"
 3108:
 3109:
```

add_files (docs/build_complete_guide.py, lines 3110-3139)

add_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3110-3139 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, get, add_table, file_type, stat, exists, is_file, lower, read_text, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3110: def add_files(doc: Document, files: list[str]) -> None:
 3111:     text_exts = {".js", ".mjs", ".cjs", ".html", ".css", ".json", ".md", ".txt", ".yml", ".yaml",
".toml", ".nsh", ""}
 3112:     for repo_file in files:
 3113:         path = REPO / repo_file
 3114:         doc.add_heading(f"File Explained: {repo_file}", level=1)
 3115:         doc.add_paragraph(FILE_DESCRIPTIONS.get(repo_file, "Tracked repository file used by the builder
or public website."))
 3116:         add_table(doc, [
 3117:             ("Type", file_type(path, repo_file)),
 3118:             ("Size", f"{path.stat().st_size:,} bytes" if path.exists() else "missing locally"),
 3119:             ("Used by", "Builder app, public site, installer, Cloudflare, or GitHub workflow depending on
 the path."),
 3120:             ("Beginner idea", "A repository is a folder Git tracks. This file is one object Git can
version, review, and publish."),
 3121:         ])
 3122:         if path.exists() and path.is_file() and path.suffix.lower() in text_exts:
 3123:             lines = path.read_text(encoding="utf-8", errors="replace").splitlines()
 3124:             doc.add_heading("Representative snippet", level=2)
 3125:             add_code(doc, "\n".join(lines[:24]) if lines else "(empty file)")
 3126:             if len(lines) > 24:
 3127:                 doc.add_paragraph(f"Only the first 24 lines are shown here. The full file has
{len(lines):,} lines and should be opened in the repository when editing.")
 3128:         else:
 3129:             doc.add_heading("Binary or asset note", level=2)
 3130:             doc.add_paragraph("This is treated as an asset. The guide explains its purpose, but the raw
binary content is not printed because raw image/icon bytes would not be useful to read in Word.")
 3131:         doc.add_heading("Safe editing rule", level=2)
 3132:         bullets(doc, [
 3133:             "Make changes on a feature branch from development.",
 3134:             "Preview or test the behavior before merging.",
 3135:             "Do not edit generated/public files by hand if the builder parser is supposed to generate
them.",
 3136:         ])
 3137:         doc.add_page_break()
 3138:
 3139:
```

add_commands (docs/build_complete_guide.py, lines 3140-3154)

add_commands named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3140-3154 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_table, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3140: def add_commands(doc: Document) -> None:
3141:     for command, explanation, why in COMMAND_LESSONS:
3142:         doc.add_heading(f"Command Explained: {command}", level=1)
3143:         add_code(doc, command)
3144:         add_table(doc, [
3145:             ("What it does", explanation),
3146:             ("Why it matters", why),
3147:             ("Where used", "Local development, GitHub Actions, builder publishing, installer testing, or
compiler execution."),
3148:             ("Beginner warning", "Run commands from the correct folder. In this project, most development
commands belong in the builder repository root."),
3149:         ])
3150:         doc.add_heading("How to read command output", level=2)
3151:         doc.add_paragraph("Command output is the program talking back to you. Success output usually
names what happened. Error output usually names a missing file, missing tool, denied permission, wrong branch,
or failed compile step.")
3152:         doc.add_page_break()
3153:
3154:
```

add_features (docs/build_complete_guide.py, lines 3155-3177)

add_features named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3155-3177 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, lower, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3155: def add_features(doc: Document, diagrams: dict[str, Path]) -> None:
3156:     for topic in FEATURE_TOPICS:
3157:         doc.add_heading(f"Builder Feature: {topic}", level=1)
3158:         if topic in {"Compile Code workspace", "HDL testbench requirement", "HDL signal scope", "Terminal
output", "Messages log", "Append code to project"}:
3159:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {topic}.")
3160:         elif topic in {"AI chat", "Cloudflare AI worker"}:
3161:             add_diagram(doc, diagrams["cloudflare-ai-flow"], f"Context diagram for {topic}.")
3162:         elif topic in {"Installer workflow", "Updater workflow", "Release tags", "Branch protection"}:
3163:             add_diagram(doc, diagrams["release-pipeline"], f"Context diagram for {topic}.")
3164:         elif topic in {"Project preview", "Full portfolio preview", "Project parser", "Asset
synchronization"}:
3165:             add_diagram(doc, diagrams["builder-to-site-files"], f"Context diagram for {topic}.")
3166:         doc.add_paragraph(f"This page explains the {topic.lower()} area of the OMB builder and website
system.")
3167:         add_table(doc, [
3168:             ("Owner view", "The builder exposes controls for creating, editing, saving, previewing, or
publishing this area."),
3169:             ("Visitor view", "The public website shows only parsed, approved output. Editing controls
stay private."),
3170:             ("Parser role", "The parser converts local builder state into website-safe data and
markup."),
3171:             ("Risk", "If this area is not saved before preview or publishing, the public site may show an
older version."),
3172:         ])
3173:         doc.add_heading("Plain-English example", level=2)
3174:         doc.add_paragraph("Think of the builder as the kitchen and the public website as the plate served
to visitors. The parser is the person plating the content neatly. Visitors should never see the prep
controls.")
3175:         doc.add_page_break()
3176:
3177:
```

add_tradeoffs (docs/build_complete_guide.py, lines 3178-3189)

add_tradeoffs named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3178-3189 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3178: def add_tradeoffs(doc: Document) -> None:
 3179:     for title, body in TRADEOFFS:
 3180:         doc.add_heading(f"Tradeoff: {title}", level=1)
 3181:         doc.add_paragraph(body)
 3182:         add_table(doc, [
 3183:             ("Benefit", "The chosen direction fits the current builder goal: owner-controlled, local-
first, preview-before-publish portfolio creation."),
 3184:             ("Cost", "Every architecture choice gives up something. Usually the cost is complexity, setup
 work, or less flexibility in another direction."),
 3185:             ("Decision rule", "Prefer the option that keeps publishing safe, previews accurate, and
editing understandable."),
 3186:         ])
 3187:         doc.add_page_break()
 3188:
 3189:
```

add_troubleshooting (docs/build_complete_guide.py, lines 3190-3202)

add_troubleshooting named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3190-3202 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, numbers, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3190: def add_troubleshooting(doc: Document) -> None:
3191:     for title, body in TROUBLESHOOTING_TOPICS:
3192:         doc.add_heading(f"Troubleshooting: {title}", level=1)
3193:         doc.add_paragraph(body)
3194:         numbers(doc, [
3195:             "Read the exact error message before changing anything.",
3196:             "Confirm whether the issue is local builder state, Git state, installer state, public website
cache, or backend service availability.",
3197:             "Fix the smallest proven cause first.",
3198:             "Save a clean checkpoint after the fix works.",
3199:         ])
3200:         doc.add_page_break()
3201:
3202:
```

add_concepts (docs/build_complete_guide.py, lines 3203-3216)

add_concepts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3203-3216 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3203: def add_concepts(doc: Document) -> None:
 3204:     for concept in CONCEPTS:
 3205:         doc.add_heading(f"Beginner Concept: {concept}", level=1)
 3206:         doc.add_paragraph(f"{concept} is one of the building blocks behind the OMB builder and portfolio
website. You do not need to memorize every term at once. Learn the role it plays, then come back when you see it
 in an error message or file name.")
 3207:         add_table(doc, [
 3208:             ("Simple meaning", "This concept helps explain how the app, website, GitHub workflow,
installer, or backend behaves."),
 3209:             ("Where it appears", "Look for it in README instructions, source files, GitHub Actions logs,
installer behavior, browser behavior, or terminal output."),
 3210:             ("How to debug it", "Ask: what is supposed to happen, what actually happened, what file or
service owns that behavior, and what changed most recently?"),
 3211:         ])
 3212:         doc.add_heading("Tiny mental model", level=2)
 3213:         doc.add_paragraph("When confused, draw boxes. Put the user on the left, the thing they click in
the middle, and the file or service that responds on the right. Most software problems become easier once the
boxes are visible.")
 3214:         doc.add_page_break()
 3215:
 3216:
```

add_closing (docs/build_complete_guide.py, lines 3217-3231)

add_closing named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3217-3231 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, numbers, and add_note. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3217: def add_closing(doc: Document) -> None:
 3218:     doc.add_heading("Final Operating Checklist", level=1)
 3219:     numbers(doc, [
 3220:         "Make feature changes on a feature or codex branch from development.",
 3221:         "Update README and the in-app guide when behavior changes.",
 3222:         "Run local syntax checks and useful smoke tests.",
 3223:         "Merge feature work into development.",
 3224:         "Only merge development into main for release-ready builds.",
 3225:         "Bump package.json before a new main release.",
 3226:         "Confirm GitHub Actions publish a fresh installer and stable latest asset.",
 3227:         "Open the builder, check update behavior, and verify the public website link downloads the latest
 installer.",
 3228:     ])
 3229:     add_note(doc, "The core rule", "The builder is the source of truth for creating content. The preview
is the truth test before publishing. GitHub is the delivery path. The public website is the final read-only
result.")
 3230:
 3231:
```

main (docs/build_complete_guide.py, lines 3232-3271)

main named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3232-3271 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references tracked_files, load_package, make_diagrams, setup_document, add_cover, add_reading_map, add_entry_points, add_architecture, add_flow_walkthroughs, add_file_communication, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3232: def main() -> None:
 3233:     files = tracked_files()
 3234:     package = load_package()
 3235:     diagrams = make_diagrams()
 3236:     doc = setup_document()
 3237:     add_cover(doc, package)
 3238:     add_reading_map(doc)
 3239:     add_entry_points(doc)
```

```
    add_architecture(doc, diagrams)
    add_flow_walkthroughs(doc, diagrams)
    add_file_communication(doc, diagrams)
    add_tool_build_map(doc, diagrams)
    add_software_decisions(doc)
    add_system_layers(doc, diagrams)
    add_data_contracts(doc)
    add_api_endpoint_catalog(doc)
    add_programming_syntax(doc)
    add_shell_syntax(doc)
    add_caching_methods(doc, diagrams)
    add_installer_details(doc, diagrams)
    add_function_maps(doc, diagrams)
    add_complete_function_inventory(doc, files)
    add_generated_repository_file_reference(doc, files, diagrams)
    add_generated_code_reference(doc, files, diagrams)
    add_generated_file_reference(doc, files, diagrams)
    add_deep_detail_topics(doc)
    add_workflows(doc, files, diagrams)
    add_files(doc, files)
    add_commands(doc)
    add_features(doc, diagrams)
    add_tradeoffs(doc)
    add_troubleshooting(doc)
    add_concepts(doc)
    add_closing(doc)
    removed_breaks = compact_generated_docx(doc, keep_first_page_break=True)
    print(f"Compacted generated DOCX layout by removing {removed_breaks} page breaks.")
    doc.save(OUTPUT)
    print(OUTPUT)


```

Representative opening snippet

```
from __future__ import annotations

import html
import json
import math
import mimetypes
import re
import shutil
import subprocess
import textwrap
from collections import Counter
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont
from docx import Document
from docx.enum.table import WD_CELL_VERTICAL_ALIGNMENT, WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Inches, Pt, RGBColor

try:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import LETTER
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import inch
    from reportlab.platypus import Image as PdfImage
    from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer, Table,
TableStyle

    REPORTLAB_AVAILABLE = True
except ImportError:
    REPORTLAB_AVAILABLE = False


REPO = Path(__file__).resolve().parents[1]
OUTPUT = REPO / "docs" / "OMB_Portfolio_Builder_Complete_Guide.docx"
DIAGRAM_DIR = REPO / "docs" / "guide-diagrams"
CODE_REFERENCE_DIR = REPO / "docs" / "code-reference"
CODE_REFERENCE_DOCX_DIR = REPO / "docs" / "code-reference-docx"
CODE_REFERENCE_PDF_DIR = REPO / "docs" / "code-reference-pdf"
CODE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.docx"
CODE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.pdf"
ALL_FILE_REFERENCE_DOCX_DIR = REPO / "docs" / "file-reference-docx"
ALL_FILE_REFERENCE_PDF_DIR = REPO / "docs" / "file-reference-pdf"
ALL_FILE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_File_Reference.docx"
ALL_FILE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_File_Reference.pdf"
FILE_REFERENCE_DOCX_DIR = REPO / "docs" / "important-code-reference-docx"
FILE_REFERENCE_PDF_DIR = REPO / "docs" / "important-code-reference-pdf"
FILE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_Important_Code_Reference.docx"
FILE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_Important_Code_Reference.pdf"


FILE_DESCRIPTIONS = {
    ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows
```

```
installer and portable application, checks the stable download link, enforces release version bumps, uploads
artifacts, and publishes GitHub Releases.",
   55:     ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main
and rejects pull requests into main unless the source branch is development.",
   56:     ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should
not be committed.",
   57:     ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files
publish exactly as generated.",
   58:     ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners
 to find responsible disclosure information.",
   59:     "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and
release flow.",
   60:     "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are
present.",
   61:     "README.md": "Main README for the standalone builder application: install, updates, workspaces, text
editing, publishing, build commands, branch model, and uninstall.",
   62:     "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public
website.",
   63:     "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or
compatible launch surfaces.",
   64:     "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
   65:     "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
   66:     "assets/favicon-32.png": "Standard browser favicon size.",
   67:     "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
   68:     "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
   69:     "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
   70:     "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
   71:     "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
   72:     "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.